B1GMB42 Software Manual

IndianaDell workspace

2026-09-04

B1GMB42 Software Manual

Machine: Dell Precision Tower 5810 (B1GMB42)
Hostname: Tower5810
OS: Ubuntu 26.04 LTS (resolute)
Workspace: `~/Documents/IndianaDell`

Companion hardware manual: `B1GMB42-slot-port-inventory.md` (slots, GPUs, storage, PERC, ports, lab USB dock)
Lab USB dock (WCH + CS202 audio + Genesys microSD): hardware manual USB section + `docs/usb-wch-cs202-dock.md`
Lab host Thumper (NVIDIA GPUs, power/clock locks): `docs/thumper-gpu.md`
Lab host Thumper (ZFS / disks / boot reconstruction): `docs/thumper-storage.md`

This manual documents every **host-facing install** the IndianaDell workspace provides: apt packages, rustup, Python venvs, built tools, Flatpak apps, GNOME preferences, Plymouth themes, optional GPU/ROCm tooling, ZFS recovery, Ventoy live persistence, and GitHub sync. Each chapter covers one topic using the same structure:

1. What gets installed
2. How it is installed
3. How to verify
4. How to customize
5. What `bin/rebuild-machine` does and does not do

Build PDFs: `bin/build-all-docs` (all manuals) or `bin/build-software-manual` (this book only).

Quick reference: `docs/features-available.md` (cheat sheet, not a replacement for this manual).

Chapter index (this manual): - 00 Front matter - 01 Introduction - 02 Rebuild and recovery - 03 Post-rebuild checklist - 04 Development - 05 Themes - 06 GPU and display - 07 GNOME session - 08 GNU Radio / SDR - 09 Ham radio - 10 HackRF Mayhem - 11 Flatpak apps - 12 Machine utilities - 13 Fac-

tory docs - 14 Gaps and limits - 15 Ventoy live session - 16 QEMU - 17 ZFS SMB share (`/home`) - 18 Disk IO tuning (UI responsiveness) - Appendix A — Bin launchers - Appendix B — Apt packages

GitHub: https://github.com/webaugur/IndianaDell (private)

Supersedes: flat `B1GMB42-software-inventory.md` (now a stub with links here).

Chapter 1 — Introduction

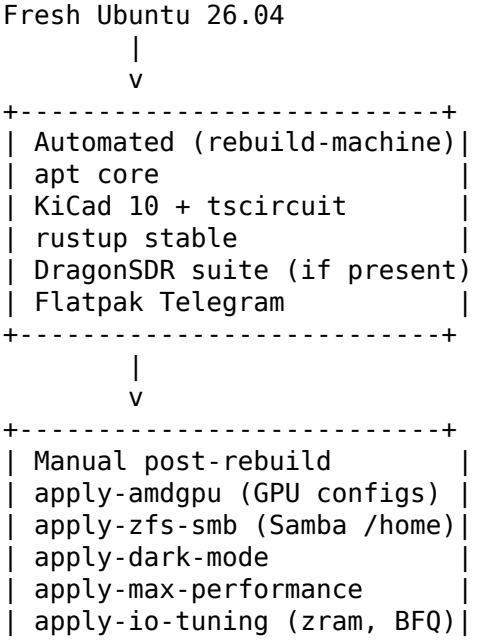
What this workspace installs

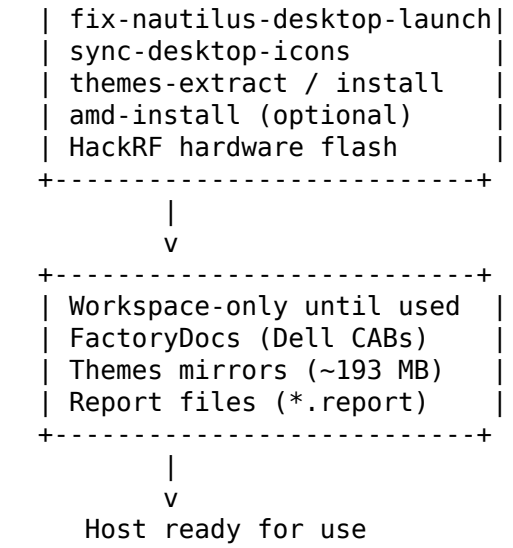
IndianaDell is a **software restoration toolkit** for Tower5810 after a fresh Ubuntu 26.04 install. It does not partition disks, configure ZFS, flash BIOS, or install Windows. It restores the development, SDR, ham radio, HackRF/Mayhem, documentation, and workstation utility stack documented in the chapters that follow.

The workspace lives at `~/Documents/IndianaDell`. Copy or clone it before running `bin/rebuild-machine`.

Install layering

Software arrives in three layers. Understanding the order prevents skipped steps after a reinstall.





Automated steps run via `bin/rebuild-machine` (see Chapter 2). **Manual** steps are intentional: GPU session files, GNOME gsettings, Plymouth overlay, and hardware flashing need user context or sudo at the right time. **Workspace-only** content stays in the repo until you invoke the matching `bin/` launcher.

Workspace vs host paths

Location	Role
`~/Documents/IndianaDell/`	Source of truth for scripts, themes, HackRF assets
`/usr/`	Apt-installed binaries, Plymouth themes, udev rules (after apply)
`~/.cargo/`	Rust toolchain (rustup)
`~/.local/`	tscircuit / `tsci` (npm `--prefix`, KiCad TypeScript companion)
`hackrf/venv-urh/`	Universal Radio Hacker Python venv
`hackrf/build/`	HackRF host tools built from source
`hackrf/local/`	Optional CMAKE_INSTALL_PREFIX for built libhackrf
`Themes/*/mirror/`	Frozen copies of apt-owned theme files

Location	Role
`FactoryDocs/`	Dell vendor packages (not auto-installed to host)

Reading guide

If you need…	Read
Full restore after reinstall	Ch. 2 + Ch. 3
KiCad 10, tscircuit (`tsci` TypeScript → `.kicad_sch`)	Ch. 2 + Ch. 3 + Appendix B
Python, Rust, pandoc, Node	Ch. 4
Boot/login/desktop look	Ch. 5 + Ch. 7
FirePro GPUs, ROCm	Ch. 6
GNU Radio, gqrx, SoapySDR	Ch. 8
fldigi, WSJT-X, CHIRP	Ch. 9
HackRF, Mayhem, URH	Ch. 10
Lab USB dock (audio + microSD)	Hardware manual USB section; `docs/usb-wch-cs202-dock.md`
Telegram	Ch. 11
iotest, dellmerge	Ch. 12
Dell driver CABs	Ch. 13
Known gaps	Ch. 14
Ventoy live persistence (Uncle Wiggly + PNY rebuild stick), Grok, secrets	Ch. 15
ZFS `rpool` / `bpool` recovery	Ch. 2 + `docs/B1GMB42-zfs-recovery.md` (+ Ch. 15 live boot)
`/etc/default/zfs` force import	Ch. 2 / ZFS recovery manual — `ZPOOL_IMPORT_OPTS="-f"`
All `bin/` commands	Appendix A
All apt package names	Appendix B

PATH and launchers

IndianaDell `bin/` and `scripts/` directories are prepended to `PATH` via `~/.config/indianadell/path.sh` (sourced from `~/.bashrc`). Project tools override same-named system binaries.

Related documents

- **Hardware:** `B1GMB42-slot-port-inventory.md` + PDF — GPUs, PERC, bays, ports, lab USB dock
- **Lab USB dock:** WCH hubs `1a86:8095`, CS202/AB13X audio `001f:0b21`, Genesys microSD `05e3:0751` — hardware manual + `docs/usb-wch-cs202-dock.md` (not the JMicron SATA dock)
- **ZFS recovery:** `docs/B1GMB42-zfs-recovery.md` + PDF — live-media rpool/bpool chroot
- **PERC IT flash:** `docs/B1GMB42-perc-it-flash.md` — H710 FreeDOS/Wiggly path
- **Themes deep-dive:** `Themes/README.md` and per-folder READMEs
- **HackRF inventory:** `hackrf/MANIFEST.txt`
- **Apt snapshots:** `apt-full-manifest.txt`, `apt-hamradio-dev-manifest.txt`
- **Rebuild log:** `scripts/rebuild/last-run.log`

Chapter 2 — Rebuild and Recovery

What gets installed

`bin/rebuild-machine` restores the **workstation** software stack (core apt, **KiCad 10**, **samsungtv** LAN remote, rustup, Flatpak Telegram). The **SDR / ham / HackRF** stack is installed from **DragonSDR** when `~/Documents/DragonSDR` is present (`bin/install-dragonsdr`).

How it is installed

```
cd ~/Documents/IndianaDell
chmod +x bin/* scripts/rebuild/*.sh
bin/rebuild-machine                 # full restore (includes DragonSDR suite if pr
bin/rebuild-machine --verify-only   # check only, no installs
bin/install-dragonsdr               # SDR suite alone
```

Environment overrides:

Variable	Effect
`SKIP_TELEGRAM=1`	Skip Flatpak Telegram install
`SKIP_DRAGONSDR=1`	Skip SDR suite install/verify
`SKIP_HACKRF_BUILD=1`	Forwarded to DragonSDR suite (skip cmake host build)
`SKIP_HAM=1`	Forwarded to DragonSDR (skip desktop ham apps)

Variable	Effect
`SKIP_KICAD=1`	Skip KiCad 10 PPA + `APT_KICAD` + tscircuit npm (ngspice, gerbv, 3D libs)
`SKIP_SAMSUNGTV=1`	Skip `samsungtv` CLI (`pipx install samsungtvws[cli]`)
`DRAGONSDR_ROOT=…`	Override suite path (default `~/Documents/DragonSDR`)

Phases (from `scripts/rebuild/rebuild-machine.sh`):

Phase	Action
1	`apt-get update`
2	Install `APT_CORE` — build, Python, docs, GPU utils, flatpak, gh
2b	KiCad 10 PPA + `APT_KICAD` + tscircuit npm (unless `SKIP_KICAD=1`)
2c	`samsungtv` via pipx `samsungtvws[cli]` (unless `SKIP_SAMSUNGTV=1`)
3	Flatpak remote + `org.telegram.desktop` (unless skipped)
4	rustup stable if `rustc` missing
5	DragonSDR `install-suite` (apt SDR/ham + HackRF/Mayhem/URH) unless skipped
6	chmod `bin/` and scripts
7	Regenerate apt manifests; run `verify_stack`

Log file: `scripts/rebuild/last-run.log`

How to verify

```
bin/rebuild-machine --verify-only
bin/install-dragonsdr --verify-only
```

`verify_stack` checks:

- Every package in `APT_CORE` via `dpkg-query`
- Every package in `APT_KICAD` plus `kicad` / `kicad-cli` / `ngspice` / `node` / `npm`, `import pcbnew` 10.*, and tscircuit + `@tscircuit/capacity-autorouter` (unless `SKIP_KICAD=1`)

- samsungtv pipx package samsungtvws[cli] (unless SKIP_SAMSUNGTV=1)
- Commands: rustc, cargo, pandoc, xelatex, vkcube
- Launchers: dellmerge, gpu-stress, iotest, apply-amdgpu, rebuild-machine
- DragonSDR suite (unless SKIP_DRAGONSDR=1 or suite missing)
- Flatpak Telegram (unless SKIP_TELEGRAM=1)

Exit code 0 means all checks passed.

How to customize

- **Add workstation apt packages:** Edit scripts/rebuild/package-lists.sh, update Appendix B, re-run rebuild.
- **Add SDR/ham packages:** Edit ~/Documents/DragonSDR/tools/package-lists.sh, re-run bin/install-dragonsdr.
- **Pin Mayhem version:** Edit DragonSDR/hackrf/scripts/download-mayhem.sh.
- **Skip heavy steps:** Use SKIP_* env vars for CI or partial recovery.

What rebuild does / does not do

Rebuild **does**	Rebuild **does not**
apt install APT_CORE + APT_KICAD (KiCad 10 PPA)	Partition disks or ZFS
tscircuit + @tscircuit/capacity-autorouter under ~/.local	sudo bin/apply-amdgpu
samsungtv (pipx install samsungtvws[cli]) unless SKIP_SAMSUNGTV=1	Pair the TV (on-screen Allow)
rustup; DragonSDR suite when present	GNOME prefs / themes by default
Flatpak Telegram	Flash HackRF / PortaPack firmware
Regenerate apt manifests	bin/amd-install (ROCm)
chmod workspace scripts	Install FactoryDocs CABs to Windows

After a successful rebuild, continue with **Chapter 3 — Post-Rebuild Checklist**.

ZFS rpool / bpool recovery

Full guide: docs/B1GMB42-zfs-recovery.md + B1GMB42-zfs-recovery.pdf (also on **DOSBOOT** IndianaDell/recovery/).

Required on the installed system: `/etc/default/zfs` must set force import so boot can recover after export or unclean shutdown:

```
# /etc/default/zfs
ZPOOL_IMPORT_OPTS="-f"
```

Verify anytime: `grep ZPOOL_IMPORT_OPTS /etc/default/zfs`. One-shot boot alternative: kernel cmdline `zfsforce=1`.

Quick (with scripts, from Ventoy live):

```
sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell    # or DOSBOOT/IndianaDell/recovery
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
```

Chapter 3 — Post-Rebuild Checklist

`bin/rebuild-machine` intentionally stops before steps that need a logged-in desktop, a reboot, or hardware attached. Run this checklist once per fresh install.

1. GPU session configuration

Tower5810 has three AMD FirePro cards (W5000/W5100). Multi-GPU Wayland/X11 configs live in `etc/`.

```
cd ~/Documents/IndianaDell
sudo bin/apply-amdgpu
sudo reboot
```

Verify after reboot: `echo $WAYLAND_DISPLAY`, `glxinfo -B`, `vkcube` on each display if needed.

See Chapter 6 for ROCm (`bin/amd-install`) — optional and not supported for ML on these GPUs.

```
sudo bin/apply-sensor-watch          # thermal/fan watchdog (delayed systemd)
indiana-sensor-watch --once          # print current sensors
```

2. GNOME session preferences

Run as the **desktop user** (not root):

```
bin/apply-dark-mode                  # Yaru-dark GTK, shell, icons, GDM greeter
bin/apply-max-performance            # no suspend, dimming, or night light
bin/fix-nautilus-desktop-launch     # Nautilus 50+: double-click .desktop launches
bin/sync-desktop-icons               # Nautilus 50+: show Icon= as file icon
```

Verify:

```
gsettings get org.gnome.desktop.interface color-scheme
powerprofilesctl get
bin/fix-nautilus-desktop-launch --status   # expect application/x-desktop → xdg-de
bin/sync-desktop-icons -v                   # sets metadata::custom-icon* from Ico
```

See Chapter 7 for every gsettings key touched, the Nautilus 50 "Allow Launching" note, and custom-icon metadata.

3. Boot splash (optional)

Default Ubuntu **bgrt** Plymouth theme shows Dell BGRT center + Ubuntu watermark. To customize:

```
bin/themes-extract                    # refresh mirrors + extract logos
# edit Themes/boot/overlay/watermark.png or background.png
sudo bin/themes-install-boot          # or --oem / --no-watermark
sudo reboot
```

Restore factory: `sudo bin/themes-restore-boot`

See Chapter 5.

4. HackRF hardware (when device is available)

```
bin/install-dragonsdr --verify-only   # suite present?
source bin/hackrf-env                 # PATH → ~/Documents/DragonSDR/hackrf
hackrf_info                           # should list board
bin/hackrf-flash-mayhem               # extract USB flash bundle
# follow bundle README for DFU flash
bin/hackrf-prepare-sdcard             # ensure SD tree is extracted
# copy ~/Documents/DragonSDR/hackrf/sd-card/mayhem-v2.4.0/* to FAT32 microSD root
```

Verify: `hackrf_info`, on-device Mayhem version, SD apps visible on PortaPack.

See Chapter 10 and `~/Documents/DragonSDR/README.md`.

5. KiCad 10 and tscircuit (workstation default)

Installed by `bin/rebuild-machine` Phase 2b unless `SKIP_KICAD=1`. Opt out is rebuild-time only; this checklist just confirms the GUI/CLI stack.

KiCad 10 (PPA `kicad/kicad-10.0-releases`): schematic/PCB editor, 3D models, `ngspice`, `gerbv`.

tscircuit (github.com/tscircuit/tscircuit): TypeScript/React circuits via `tsci`. Export to KiCad, then open the files in KiCad 10. Autorouter package is `@tscircuit/capacity-autorouter` (tscircuit-autorouter), installed under `~/.local` with `tsci`.

```
kicad-cli version
python3 -c 'import pcbnew; print(pcbnew.Version())'
command -v ngspice gerbv gerbview tsci
tsci --help | head
```

5b. samsungtv (workstation default)

Installed by `bin/rebuild-machine` Phase 2c unless `SKIP_SAMSUNGTV=1`. LAN remote for the Tower5810 Samsung (UN32M4500, `10.0.0.31`). First run: Allow the client on the TV.

```
samsungtv device-info
samsungtv home
```

TypeScript KiCad:

```
tsci init my-board
cd my-board
tsci export index.tsx -f kicad_sch    # also kicad_pcb, kicad_zip, kicad-library
```

Expect `10.0.*` from `kicad-cli` and pcbnew. `kicad-doc-id` may remain on 9.x; that is not a blocker.

6. Documentation PDFs

```
bin/build-all-docs                    # software manual + hardware + inventory PDF
# or:
bin/build-software-manual             # this manual only
```

Outputs: `B1GMB42-software-manual.pdf`, `B1GMB42-slot-port-inventory.pdf`, `B1GMB42-software-inventory.pdf`.

7. Machine inventory baseline

```
bin/dellmerge > b1gmb42.report
sudo bin/iotest                       # optional storage survey
bin/gpu-stress 60 vkcube              # optional GPU smoke test
```

8. FactoryDocs recovery (optional)

Only 19 of 101 pre-crash Dell packages are on disk. Re-download per `FactoryDocs/README.md` and `MANIFEST-pre-crash.txt`. These are **workspace archives**, not installed by rebuild.

See Chapter 13.

9. ZFS SMB share (`/home`)

Samba is in `APT_CORE`. After rebuild, bind it to the LAN and publish the home dataset (not `/root`):

```
sudo bin/apply-zfs-smb
sudo smbpasswd -a user
zfs get -r sharesmb rpool/USERDATA   # only home_ulmkkv on
ss -lnt | grep ':445'                # 10.0.0.113 and 127.0.0.1
```

See Chapter 17. Do not leave Ubuntu's inherited `sharesmb=on` on `rpool/USERDATA` — that would export `/root`.

10. ZFS force import (required on this host)

After any reinstall or recovery chroot, confirm the installed system will force-import pools at boot:

```
grep '^ZPOOL_IMPORT_OPTS' /etc/default/zfs
# must show: ZPOOL_IMPORT_OPTS="-f"
```

If missing, set it in `/etc/default/zfs`, then `sudo update-initramfs -c -k all`. Without this, boot can hang after a recovery export or unclean shutdown. See Chapter 2 and `docs/B1GMB42-zfs-recovery.md`.

Quick verification block

```
cd ~/Documents/IndianaDell
bin/rebuild-machine --verify-only
kicad-cli version                                  # expect 10.0.*
tsci --help >/dev/null && echo OK tscircuit   # TypeScript → KiCad
grep '^ZPOOL_IMPORT_OPTS' /etc/default/zfs    # expect "-f"
zfs get -H -o name,value sharesmb rpool/USERDATA/home_ulmkkv  # on
source bin/hackrf-env
. ~/.cargo/env && rustc --version
gnuradio-config-info --version
bin/urh --version
```

Summary table

Step	Command	Reboot?
GPU configs	`sudo bin/apply-amdgpu`	Yes
Dark mode	`bin/apply-dark-mode`	No
Max performance	`bin/apply-max-performance`	No

Step	Command	Reboot?
Nautilus 50 .desktop launch	`bin/fix-nautilus-desktop-launch`	No
Nautilus 50 .desktop icons	`bin/sync-desktop-icons`	No
Custom boot	`sudo bin/themes-install-boot`	Yes
HackRF flash	`bin/hackrf-flash-mayhem` + DFU	Maybe
KiCad 10 + tscircuit	`kicad-cli version`; `tsci export … -f kicad_sch`	No
ROCm (optional)	`bin/amd-install`	Yes
All doc PDFs	`bin/build-all-docs`	No
Disk IO tuning + capped Chrome	`sudo bin/apply-io-tuning`; `bin/apply-chrome-cap` (Ch. 18)	No
ZFS SMB `/home`	`sudo bin/apply-zfs-smb`; `sudo smbpasswd -a user`	No
ZFS force import	check `/etc/default/zfs` □ `ZPOOL_IMPORT_OPTS="-f"`	If initramfs updated
Ventoy persistence seed	`~/bin/seed-ventoy-persistence.sh`	No

Chapter 4 — Development Toolchain

What gets installed

Component	Version / path	Install source
Python	3.14 (system)	Ubuntu base + apt
pip, venv	apt	`python3-pip`, `python3-venv`
numpy, scipy, matplotlib	apt	science stack
PyQt5	apt	GNU Radio Companion, some tools
Rust	1.96.1 stable	rustup to `~/.cargo/bin`

Component	Version / path	Install source
C/C++ build	apt	`build-essential`, `cmake`, `pkg-config`
Clang/LLVM	apt	bindgen-style Rust, native tooling
SSL/USB/FFTW/Volk	apt dev libs	SDR and native builds
ARM cross-compile	apt	PortaPack Mayhem firmware (`gcc-arm-none-eabi`)
pandoc + XeLaTeX	apt	Manual PDF generation
Git, curl, wget	apt	repos and downloads
GitHub CLI (gh)	2.46 (apt)	HTTPS/token fallback; `gh auth login` for API access
Node.js + npm	apt (APT_KICAD)	tscircuit CLI
tscircuit (`tsci`)	npm `--prefix ~/.local`	TypeScript/React → `.kicad_sch` / `.kicad_pcb` (Chapter 3)
`@tscircuit/capacitor-autorouter`	npm `--prefix ~/.local`	PCB autorouter used by tscircuit

Python bindings verified on this host: `gnuradio`, `SoapySDR`, `Hamlib` (capital H in Python).

Project venv: URH lives in `~/Documents/DragonSDR/hackrf/venv-urh/` (Chapter 10), not system-wide.

How it is installed

Apt (rebuild Phase 2): packages listed in Appendix B under "Development" and shared dev libraries.

Rust (rebuild Phase 5):

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y --default-
source ~/.cargo/env
```

Rebuild skips rustup if `rustc` is already on PATH.

How to verify

```
python3 --version
python3 -c "import numpy, scipy, matplotlib; print('OK')"
. ~/.cargo/env && rustc --version && cargo --version
cmake --version | head -1
```

```
arm-none-eabi-gcc --version | head -1
pandoc --version | head -1
xelatex --version | head -1
node --version && npm --version
tsci --help >/dev/null && echo OK tscircuit
```

How to customize

- **New Python project venv:** `python3 -m venv myproject/.venv && . myproject/.venv/bin/activate`
- **Rust toolchain:** `rustup toolchain install stable`, `rustup component add clippy`
- **Per-project Rust SDR crates:** `cargo add rtlsdr` etc. — no workspace scaffold ships with IndianaDell
- **Manual PDFs:** `bin/build-all-docs` (all docs) or `bin/build-software-manual` (software manual only)

What rebuild does / does not do

Does	Does not
Install all dev apt packages	Create application-specific venvs beyond URH
Install rustup if missing	Pin a non-stable Rust toolchain
chmod workspace scripts	Install IDE or editor plugins

Chapter 5 — Themes (Boot, Login, Desktop)

What gets installed

The **Themes/** module (~194 MB with mirrors) documents and customizes three visual stages:

Stage	What you see	Apt packages	Workspace folder
Boot	Dell BGRT center + spinner + Ubuntu watermark	`plymouth`, `plymouth-theme-spinner`, …	`Themes/boot/`
Login	GDM on GNOME Shell	`gdm3`, `gnome-shell`, `yaru-theme-gnome-shell`	`Themes/login/`

Stage	What you see	Apt packages	Workspace folder
Desktop	Yaru GTK, icons, shell	`yaru-theme-gtk`, `yaru-theme-icon`, …	`Themes/desktop/`

Active Plymouth theme: `bgrt` at `/usr/share/plymouth/themes/bgrt/bgrt.plymouth`

Extracted boot logos:

- `Themes/boot/extracted/bgrt-firmware-oem.png` — Dell from UEFI BGRT (`/sys/firmware/acpi/bgrt/image`)
- `Themes/boot/extracted/ubuntu-watermark-dark.png` — bottom Ubuntu text

Custom Plymouth install target: `indianadell` theme under `/usr/share/plymouth/themes/indian`

Each subfolder has its own `README.md` and `apt-packages.txt`. See `Themes/MANIFEST.txt` for a one-page map.

How it is installed

Themes are **not** applied by `bin/rebuild-machine`. Use launchers:

```
bin/themes-extract              # snapshot apt-owned files + extract logos (~193 M
sudo bin/themes-install-boot    # install custom Plymouth from boot/overlay/
sudo bin/themes-restore-boot    # revert to stock bgrt
bin/apply-dark-mode             # login + desktop dark (Chapter 7)
```

Boot overlay workflow:

```
cp my-logo.png Themes/boot/overlay/watermark.png    # bottom Ubuntu text only
sudo bin/themes-install-boot

cp my-splash.png Themes/boot/overlay/background.png
sudo bin/themes-install-boot --oem                  # replace center (disable BGRT

sudo bin/themes-install-boot --no-watermark         # hide bottom logo
```

Every Plymouth install runs `update-initramfs -u` — **reboot required** to preview.

How to verify

```
plymouth-set-default-theme -l
plymouth-set-default-theme                      # shows active theme
ls Themes/boot/extracted/
gsettings get org.gnome.desktop.interface gtk-theme   # after apply-dark-mode
```

How to customize

Deep documentation lives in module READMEs — this chapter summarizes:

- **Boot:** `Themes/boot/README.md` — overlay/, stock/, indianadell/ plymouth definition
- **Login:** `Themes/login/README.md` — GDM greeter assets mirrored from apt
- **Desktop:** `Themes/desktop/README.md` — GTK/icon/shell Yaru variants

Center Dell logo comes from BIOS BGRT firmware, not an Ubuntu file. Change it in BIOS setup or use `--oem` with your PNG.

What rebuild does / does not do

Does	Does not
Install `plymouth`, `gdm3`, Yaru via apt (Phase 2–3)	Run `themes-extract` or `themes-install-boot`
	Set dark mode or GDM greeter prefs
	Copy overlay PNGs into initramfs

Run Chapter 3 checklist items 2 and 3 after rebuild.

Chapter 6 — GPU and Display

What gets installed

Component	Source	Purpose
`vulkan-tools`, `mesa-utils`, `clinfo`	apt (APT_CORE)	Vulkan/OpenGL/OpenCL diagnostics
`etc/` multi-GPU configs	workspace	Wayland, X11, udev, GDM tweaks
`amd-radeon/` scripts	workspace	Optional ROCm driver install
`bin/gpu-stress`	workspace	3-GPU Vulkan smoke test

Hardware (this machine): 2x AMD FirePro W5000 + 1x FirePro W5100. Vulkan and OpenCL work for graphics/compute smoke tests. **ROCm ML/HIP is not supported** on these cards (see Chapter 14).

How it is installed

Apt (automated): GPU utility packages install during rebuild Phase 2.

Session configs (manual):

```
sudo bin/apply-amdgpu    # runs etc/apply.sh
sudo reboot
```

`etc/apply.sh` installs:

- `etc/environment.d/99-amdgpu-wayland.conf`
- `etc/X11/xorg.conf.d/20-amdgpu-multi-gpu.conf`
- `etc/modprobe.d/amdgpu-multigpu.conf` (`runpm=0`)
- `etc/udev/rules.d/99-amdgpu-multigpu.rules` (tags + DPM auto hook)
- `etc/amdgpu-set-dpm-auto.sh` `/usr/local/sbin/indiana-amdgpu-dpm-auto`
- `etc/profile.d/amdgpu-multigpu.sh`
- `etc/gdm3/custom.conf` (if present)

DPM auto (all cards): every amdgpu uses load-based clocks (`auto` / `balanced`). These FirePro W5000/W5100 overheat when pinned to `high`. `apply-amdgpu` runs the helper immediately; udev re-applies when cards appear at boot.

```
# verify
for c in /sys/class/drm/card[0-9]/device; do
  [[ -f $c/power_dpm_force_performance_level ]] || continue
  echo "$(basename $(dirname $c)): level=$(cat $c/power_dpm_force_performance_leve
done
# expect: level=auto  state=balanced  on card1..card3
```

Do not set `high` on this machine. To force clocks only for a short smoke test: `echo high | sudo tee …/power_dpm_force_performance_level`, then `echo auto` when done.

Optional ROCm:

```
bin/amd-preflight        # check prerequisites
bin/amd-install          # full driver stack from amd-radeon/
bin/amd-verify
bin/amd-uninstall        # remove if needed
```

How to verify

```
vkcube                   # Vulkan cube (per display)
clinfo | head -30        # OpenCL platforms/devices
glxinfo -B               # OpenGL renderer
bin/gpu-stress 60 vkcube # stress all GPUs ~60s
lspci -nn | grep -i vga
```

How to customize

- Edit files under `etc/` before re-running `sudo bin/apply-amdgpu`
- ROCm install scripts and README in `amd-radeon/` — machine-specific; read preflight output
- Hardware details: `B1GMB42-slot-port-inventory.md` Video section

What rebuild does / does not do

Does	Does not
Install mesa-utils, vulkan-tools, clinfo	Run `apply-amdgpu`
Ensure `bin/gpu-stress` is executable	Install ROCm
	Configure monitor layout (use GNOME Settings)

Required post-rebuild: `sudo bin/apply-amdgpu` + reboot (Chapter 3).

Sensor watchdog

`thermald` does not run on this Haswell Xeon. `bin/apply-sensor-watch` installs `indianadell-sensor-watch.service` (starts after `graphical.target`, sleeps 120 s on a cold boot, then polls). It logs out-of-range CPU/GPU/DIMM/fan/SMART temps to syslog, pops a GNOME notification on the logged-in session bus, and steps GPU clocks / CPU `scaling_max_freq` down until under 80 °C (back up after a 72 °C dead-band). Disk SMART uses `smartctl -n standby,0` so sleeping HDDs are not woken.

A desktop notice is sent on **`systemctl reload`** and **`systemctl restart`**, not on first start or stop. Test the path without cycling the unit:

```
sudo indiana-sensor-watch --notify test
sudo systemctl reload indianadell-sensor-watch
```

```
sudo bin/apply-sensor-watch
indiana-sensor-watch --once
journalctl -t indiana-sensor-watch -f
```

Config: `/etc/indiana-sensor-watch.conf` (defaults in `etc/indiana-sensor-watch.conf`). `THROTTLE=0` / `THROTTLE_CPU=0` disable clock writes. `dell_smm` fan2 is ignored (unused HDD_FAN header).

Monitor input select

These FirePros are mini-DP only. There is no `/dev/cec`, so HDMI-CEC cannot switch the set.

`indiana-monitor-input` uses **DDC/CI** (`ddcutil`, VCP 60) on the current cable:

```
indiana-monitor-input detect
indiana-monitor-input grab    # request DisplayPort-1 (this PC)
indiana-monitor-input set hdmi1
```

Verified on this Samsung (2018) via `card2-DP-4` / i2c-7: EDID works, **I2C 0x37 does not answer**. Until the OSD "DDC/CI" option is on (or a different panel is used), `grab` will fail with that fact. User must be in group `i2c` (or use sudo).

IR HDMI select (Arduino Uno)

When DDC/CEC cannot switch the TV, a 940 nm LED on an Uno can send one Samsung discrete code. Wiring and flash: `tools/ir-blaster/README.md`.

```
indiana-ir-flash                # arduino-cli; no IDE
indiana-ir-send hdmi2
```

Host waits for READY/OK on the serial fd (Uno resets on port open). Group `dialout` required.

LAN remote (`samsungtv`)

Tizen WebSocket CLI for the Samsung on Tower5810 (UN32M4500, `10.0.0.31`). Default-on via `pipx install samsungtvws[cli]`; opt out `SKIP_SAMSUNGTV=1`. Wrapper: `bin/samsungtv` (host `10.0.0.31`, port `8002`, token `~/.config/indianadell/samsungtv.token`).

First run shows **Allow** on the TV. Then:

```
samsungtv device-info
samsungtv home
samsungtv volume-up
samsungtv send-key KEY_HDMI
samsungtv apps
samsungtv app-run 111299001912   # YouTube
```

Override host: `SAMSUNGTV_HOST=… samsungtv …` or `--host`. IR (`indiana-ir-send`) still works if Smart Hub is down. `SKIP_SAMSUNGTV=1` skips install on rebuild and `fix-indianadell`.

Lab probe notes, protocol comparison (WebSocket / UPnP / SmartThings), and recipes: `docs/samsung-un32m4500.md` · `UN32M4500-lab-manual.pdf`.

Lab host note — Thumper (NVIDIA)

This chapter is **Tower5810 / AMD only**. Lab topology: `docs/lab-hosts.md`. The intelligence host **`thumper.local`** (Dell Precision T5610) runs **NVIDIA TITAN Xp** and is **not** configured with `apply-amdgpu`. Safar's NVS 3100M (nvidia 340.108) is also out of scope here — `docs/safar-inventory.md`.

Full Thumper GPU doc (inventory, dual-card plan, **power/clock locking**, LingBot-Map pointer):

 `docs/thumper-gpu.md`

Summary: a second TITAN Xp with a weak cooler can be limited with `nvidia-smi -pl` / `-lgc` (and a oneshot systemd unit). Prefer `CUDA_VISIBLE_DEVICES` so heavy jobs stay on the well-cooled card. DragonSDR LingBot-Map lives under `~/Data/lingbot-map` on Thumper — see also `~/Documents/DragonSDR/tools/lingbot-map/README.md`.

Chapter 7 — GNOME Session

What gets installed

GNOME desktop preferences applied via gsettings (user session) and optionally GDM (login greeter). No apt packages beyond what Ubuntu desktop already provides (`gdm3`, `gnome-shell`, Yaru themes).

Scripts:

- `scripts/gnome/apply-dark-mode.sh` via `bin/apply-dark-mode`
- `scripts/gnome/apply-max-performance.sh` via `bin/apply-max-performance`
- `scripts/gnome/fix-nautilus-desktop-launch.sh` via `bin/fix-nautilus-desktop-launch`
- `scripts/gnome/sync-desktop-icons.sh` via `bin/sync-desktop-icons`

How it is installed

Run as the **logged-in desktop user**, not root:

```
bin/apply-dark-mode
bin/apply-max-performance
bin/fix-nautilus-desktop-launch   # Nautilus 50+ .desktop double-click launch
bin/sync-desktop-icons             # Nautilus 50+ show Icon= as file icon
```

apply-dark-mode

Sets:

Schema	Key	Value
`org.gnome.desktop.interface`	`color-scheme`	`prefer-dark`
`org.gnome.desktop.interface`	`gtk-theme`	`Yaru-dark` (override: `GTK_THEME=`)
`org.gnome.desktop.interface`	`icon-theme`	`Yaru-dark`
`org.gnome.shell.ubuntu`	`color-scheme`	`prefer-dark`
`org.gnome.desktop.wm.preferences`	`theme`	`Yaru-dark`
`org.gnome.settings-daemon.plugins.color`	`night-light`	disabled

With `APPLY_GDM=1` (default), also sets GDM greeter dark via `sudo dbus-run-session gsettings …`.

apply-max-performance

Sets:

- Power plugin: no suspend on AC/battery, no idle dim, lid close does nothing
- Session idle delay: 0
- Screensaver: no blanking or lock on idle
- Night light: off
- `powerprofilesctl set performance` when available

Nautilus 50 — “Allow Launching” removed

Change (GNOME Files / Nautilus 50, Ubuntu 26.04): Nautilus no longer runs FreeDesktop `.desktop` files itself. The old “Allow Launching” / trusted-launcher path is gone (security). Double-click falls through to the default handler for MIME type `application/x-desktop`, which is often a text editor (`gedit` / `gnome-text-editor`). So double-clicking `SDRPlusPlus.desktop` (or any app launcher on disk) **edits** the file instead of **starting** the app.

Fix: register a small MIME handler that launches the entry via `gio launch`:

Piece	Path
Handler app	`~/.local/share/applications/xdg-desktop-launch.desktop`
Wrapper script	`~/.local/bin/xdg-desktop-launch`

Piece	Path
MIME default	`application/x-desktop` → `xdg-desktop-launch.desktop`

Double-click path after install: **Files** → **xdg-open** → **wrapper** → **`gio launch`** → **your app**.

```
bin/fix-nautilus-desktop-launch              # install / reinstall
bin/fix-nautilus-desktop-launch --status     # check MIME + files
bin/fix-nautilus-desktop-launch --uninstall  # remove handler
```

Portable: the script is self-contained. Copy `scripts/gnome/fix-nautilus-desktop-launch.sh` to any Ubuntu/GNOME box and run as the desktop user (no root, no IndianaDell tree required).

CLI verify:

```
xdg-mime query default application/x-desktop   # expect xdg-desktop-launch.desktop
xdg-open /path/to/App.desktop                  # should start the app
~/.local/bin/xdg-desktop-launch /path/to/App.desktop
```

Nautilus 50 — generic `.desktop` icons

Change: even after launch works, Nautilus 50+ still shows the generic `application-x-desktop` MIME icon in the file view. It **ignores** the FreeDesktop `Icon=` field on the `.desktop` file for the list/icon view. Icons in the GNOME Shell app grid (from XDG application menus) are a different path and usually look correct; this problem is about **Files** showing launcher files as plain documents.

Fix: set GIO metadata that Nautilus still honors:

Attribute	When used	Example
`metadata::custom-icon`	`Icon=` is an absolute or relative file path	`file:///home/user/Applications/sdrpp.pr`
`metadata::custom-icon-name`	`Icon=` is a theme icon name	`utilities-terminal`

The script reads the first `Icon=` under `[Desktop Entry]` only (not `Icon[lang]=`, not other groups), then:

1. Absolute path (`/…`) → `metadata::custom-icon` with a `file://` URI; clear `custom-icon-name`
2. Relative path (`foo/bar.png`) → resolve relative to the `.desktop` file's directory; same as absolute
3. Theme name (no `/`) → `metadata::custom-icon-name`; clear `custom-icon`

4. Missing icon file warning and skip (does not fail the whole run)

Usage

```
bin/sync-desktop-icons                # scan default directories
bin/sync-desktop-icons -v             # log each set / skip / rename
bin/sync-desktop-icons --dry-run      # print actions, no gio set / rename
bin/sync-desktop-icons --file PATH    # one .desktop (inotify-friendly)
bin/sync-desktop-icons --dir DIR      # add/replace scan dir (repeatable)
bin/sync-desktop-icons --watch        # inotify loop (needs inotify-tools)
bin/sync-desktop-icons --clear-missing # unset custom-icon* if Icon= absent
bin/sync-desktop-icons --no-rename    # keep chrome-*-Default.desktop names
```

Default scan directories (when `--dir` is not used and `SYNC_DESKTOP_ICON_DIRS` is unset):

- `$HOME/.local/share/applications`
- `$HOME/Applications`
- `$HOME/Desktop`

Only **top-level** `*.desktop` files in each directory are processed (flat XDG apps layout and a personal `Applications` / Desktop folder). Nested trees are not walked.

Rename Chrome gibberish basenames

Chrome/Chromium PWAs create launchers like `chrome-lodlkdfmihgonocnmddehnfgiljnadcf-Default.desktop` while `Name=` is a short human label (X, YouTube). **By default** (`--rename`), if the file **and** its directory are **writable**, matching basenames are renamed to `${Name}.desktop` before icon metadata is applied.

Rule	Behavior
Pattern	`chrome-<id>-Default.desktop`, `chrome-<id>.desktop` (id 16 alnum); same for `chromium-`
Source of new name	First `Name=` under `[Desktop Entry]` only (not action groups, not `Name[lang]=`)
Not writable	Skip rename, warn, still try icon metadata
Target already exists	Skip rename, warn (no overwrite)
Disable	`--no-rename`

Example: `~/Desktop/chrome-lodlk…-Default.desktop` (`Name=X`) `~/Desktop/X.desktop`.

Environment:

Variable	Effect
`SYNC_DESKTOP_ICON_DIRS`	Colon-separated directory list (overrides defaults when set)

Dependencies: `gio` (`libglib2.0-bin`, already on Ubuntu desktop). `--watch` also needs `inotifywait` (`inotify-tools`).

Portable / PATH install: `bin/sync-desktop-icons` resolves its own path with `readlink -f`, so a symlink from `~/.local/bin/sync-desktop-icons` into the repo still finds `scripts/gnome/`. The script itself needs only `gio`; copy `scripts/gnome/sync-desktop-icons.sh` alone if you want it off-tree.

Companion: run **after** `bin/fix-nautilus-desktop-launch` so double-click starts the app **and** the file view shows the right icon. Neither script replaces the other.

CLI verify:

```
bin/sync-desktop-icons -v
# pick a launcher you care about:
gio info -a metadata::custom-icon -a metadata::custom-icon-name \
  "$HOME/Applications/SomeApp.desktop"
# open Files on that folder — icon should match Icon=
```

Watch mode example (re-apply when a launcher is saved or dropped into a scan dir):

```
bin/sync-desktop-icons --watch -v
# or a single-file handler:
inotifywait -m -e close_write,moved_to,create --include '\.desktop$' \
  "$HOME/Applications" | while read -r dir _ file; do
    bin/sync-desktop-icons --file "$dir$file"
  done
```

How to verify

```
gsettings get org.gnome.desktop.interface color-scheme
gsettings get org.gnome.desktop.interface gtk-theme
gsettings get org.gnome.settings-daemon.plugins.power sleep-inactive-ac-type
powerprofilesctl get
bin/fix-nautilus-desktop-launch --status
bin/sync-desktop-icons -v
```

Log out and back in to confirm GDM greeter if dark mode was applied. Refresh or reopen the Files window after `sync-desktop-icons` if icons do not update immediately.

How to customize

- Override themes: `GTK_THEME=Yaru-viridian bin/apply-dark-mode`
- Skip GDM: `APPLY_GDM=0 bin/apply-dark-mode`
- Revert individual keys with `gsettings reset …` or GNOME Settings app
- Login/desktop asset mirrors: `Themes/login/`, `Themes/desktop/`
- Remove .desktop launch fix: `bin/fix-nautilus-desktop-launch --uninstall`
- Extra icon scan dirs: `SYNC_DESKTOP_ICON_DIRS="$HOME/Apps:$HOME/Desktop" bin/sync-desktop-icons`
- Clear stale custom icons when `Icon=` is gone: `bin/sync-desktop-icons --clear-missing`
- Unset metadata on one file:
  `gio set -t unset PATH metadata::custom-icon`
  `gio set -t unset PATH metadata::custom-icon-name`

What rebuild does / does not do

Does	Does not
Install gdm3, gnome-shell, Yaru via apt	Change any gsettings Set performance power profile Install the Nautilus 50 .desktop MIME handler Run `sync-desktop-icons` or set custom-icon metadata

Run dark mode, max performance, `fix-nautilus-desktop-launch`, and `sync-desktop-icons` after every fresh install (Chapter 3).

Chapter 8 — GNU Radio and Desktop SDR

Ownership

GNU Radio, SoapySDR, and desktop SDR apps are installed by the **DragonSDR** suite, not by IndianaDell apt lists.

Item	Location
Suite install	`~/Documents/DragonSDR/bin/install-suite`
Package list	`~/Documents/DragonSDR/tools/package-lists.sh` (`APT_SDR`)
IndianaDell wrapper	`bin/install-dragonsdr`

Item	Location

What gets installed

Component	Packages / path
GNU Radio	`gnuradio`, `gnuradio-dev`, `gnuradio-doc`
Companion blocks	`gr-osmosdr`, `gr-limesdr`, `gr-fosphor`, `gr-air-modes`, `gr-hpsdr`, `gr-dab`, `gr-satellites`
SoapySDR	`libsoapysdr-dev`, `python3-soapysdr`, modules
Hardware libs	RTL-SDR, HackRF, Airspy, bladeRF, Lime, UHD
Desktop apps	`gqrx-sdr`, `quisk`, `inspectrum`, `hacktv`

SoapySDR modules (typical host): HackRF, RTL-SDR (osmosdr), Airspy, bladeRF, Lime, MiriSDR, HydraSDR, PlutoSDR, Red Pitaya, remote, audio, UHD.

How it is installed

```
bin/install-dragonsdr              # full suite (apt + HackRF workspace)
# or only packages:
bin/install-dragonsdr --apt-only
```

Called automatically during `bin/rebuild-machine` when `~/Documents/DragonSDR` is present (`SKIP_DRAGONSDR=1` to skip).

Typical workflow:

```
grcc myflowgraph.grc          # compile Companion graph
gqrx                          # general receiver GUI
quisk                         # transceiver GUI
inspectrum capture.cf32       # visualize IQ files
```

For HackRF-specific host tools and URH, see Chapter 10.

How to verify

```
bin/install-dragonsdr --verify-only
gnuradio-config-info --version
python3 -c "import SoapySDR; print('SoapySDR OK')"
SoapySDRUtil --info
command -v gqrx
```

Chapter 9 — Ham Radio (Desktop)

Ownership

Desktop ham applications are part of the **DragonSDR** suite (`APT_HAM` in `tools/package-lists.sh`).

What gets installed

Application	Command	Apt package	Role
fldigi	`fldigi`	`fldigi`	Digital modes (PSK, RTTY, …)
WSJT-X	`wsjtx`	`wsjtx`, `wsjtx-data`	FT8, JT65, weak-signal
CHIRP	`chirpw`, `chirpc`	`chirp`	Radio programming
direwolf	`direwolf`	`direwolf`	Sound-card TNC / APRS
gpredict	`gpredict`	`gpredict`	Satellite pass prediction
grig	`grig`	`grig`	Hamlib rig control GUI
xastir	`xastir`	`xastir`, `xastir-data`	APRS map client
Hamlib	API	`libhamlib-dev`, `libhamlib-utils`, `python3-hamlib`	Rig control library

How it is installed

```
bin/install-dragonsdr
# omit ham apps:
SKIP_HAM=1 bin/install-dragonsdr
```

xastir debconf: suite install preseeds `xastir/install-setuid boolean false` to avoid interactive hangs.

```
fldigi &
wsjtx &
chirpw &
direwolf -p
gpredict &
grig &
xastir &
```

How to verify

```
command -v fldigi wsjtx chirpw direwolf gpredict grig xastir
python3 -c "import Hamlib; print('Hamlib OK')"
bin/install-dragonsdr --verify-only
```

Configure rig control in each app via Hamlib model selection.

Chapter 10 — HackRF and PortaPack Mayhem

Ownership

The HackRF / PortaPack Mayhem workspace moved out of IndianaDell into **DragonSDR**:

Item	Path
Workspace	`~/Documents/DragonSDR/hackrf/`
Manifest	`~/Documents/DragonSDR/hackrf/MANIFEST.txt`
Suite install	`~/Documents/DragonSDR/bin/install-suite`
IndianaDell wrappers	`bin/hackrf-*`, `bin/urh`, `bin/install-dragonsdr`

What gets installed

Recommended firmware: PortaPack Mayhem v2.4.0

Apt packages

`hackrf`, `hackrf-firmware`, `libhackrf-dev`, `hackrf-doc`, `inspectrum`, `hacktv`, `dfu-util`, `openocd`, ARM GCC toolchain, plus GNU Radio/SoapySDR deps (Chapter 8).

Built from source (`DragonSDR/hackrf/build/`)

Tool	Notes
`hackrf_sweep`	Spectrum sweep
`hackrf_info`, `hackrf_transfer`, …	Host utilities
`libhackrf.so`	Under `hackrf/build/libhackrf/src/`

Install prefix: `hackrf/local/` (CMAKE_INSTALL_PREFIX).

Release assets (`hackrf/releases/`)

File	Purpose
`FIRMWARE_mayhem_v2.4.0.zip`	USB flash bundle
`COPY_TO_SDCARD_hackrf_mayhem_v2.4.0-no-world-map.zip`	PortaPack microSD
`OCI_hackrf_mayhem_v2.4.0.tar`	Web flasher image

Extracted SD tree: `hackrf/sd-card/mayhem-v2.4.0/`

Source repos (`hackrf/repos/`)

`hackrf`, `mayhem-firmware` (+ submodules), `portapack-hackrf`, `urh`, `hacktv`

Python venv

`hackrf/venv-urh/` — Universal Radio Hacker. Launch: `bin/urh` (wrapper DragonSDR).

udev

`hackrf/scripts/99-hackrf.rules` `/etc/udev/rules.d/`

How it is installed

```
bin/install-dragonsdr                  # full suite
SKIP_HACKRF_BUILD=1 bin/install-dragonsdr
bin/install-dragonsdr --hackrf-only   # workspace only (apt already done)
```

Manual (hardware present):

```
source bin/hackrf-env                    # PATH → DragonSDR/hackrf/build
bin/hackrf-flash-mayhem
bin/hackrf-prepare-sdcard
bin/hackrf-build-mayhem                  # compile Mayhem from source
bin/hackrf-download-mayhem
bin/urh
```

How to verify

```
bin/install-dragonsdr --verify-only
source bin/hackrf-env
hackrf_info
ls ~/Documents/DragonSDR/hackrf/sd-card/mayhem-v2.4.0/APPS | wc -l
bin/urh --version
```

Host card reader (lab USB dock)

IndianaDell machines share a USB 2 dock with a **Genesys Logic `05e3:0751`** microSD slot (enumerates only with a card in). On 2026-09-04 a PortaPack Mayhem card (FAT32 label **HackRF**, UUID `300C-16B0`) auto-mounted at `/run/media/user/HackRF`.

Use `/dev/disk/by-id/usb-Generic_STORAGE_DEVICE-0:0`, not `sdd`. The reader sat on a USB 2 daisy-chain; the kernel logged a dirty FAT volume and a brief disconnect on write. Run `fsck.vfat` before putting that card back in a PortaPack. Dock IDs, audio jack, and topology are in the **hardware manual** USB section (`B1GMB42-slot-port-inventory.md`) and `docs/usb-wch-cs202-dock.md`.

`bin/hackrf-prepare-sdcard` still writes a prepared tree; this dock is just the host slot to mount or image that card.

Chapter 11 — Flatpak Applications

What gets installed

App	Flatpak ID	Version (this host)
Telegram	`org.telegram.desktop`	6.9.3

Runtime dependency: `flatpak` package from `APT_CORE`.

How it is installed

Rebuild Phase 4:

```
flatpak remote-add --if-not-exists flathub https://dl.flathub.org/repo/flathub.fla
flatpak install -y flathub org.telegram.desktop
```

Skip with `SKIP_TELEGRAM=1 bin/rebuild-machine`.

How to verify

```
flatpak list --app | grep telegram
flatpak run org.telegram.desktop --version 2>/dev/null || true
bin/rebuild-machine --verify-only
```

How to customize

```
flatpak update org.telegram.desktop
flatpak override --user org.telegram.desktop …   # permissions, env
```

`bin/apply-dark-mode` sets `prefer-dark` color scheme; Flatpak GTK4 apps pick this up via portal when supported.

What rebuild does / does not do

Does	Does not
Install `flatpak` via apt	Install SDRangel, SigDigger (not on Flathub)
Add flathub remote + Telegram	Pin Telegram to a specific commit
Treat Telegram miss as non-fatal on install (warns in log)	Install other Flatpak apps by default

Chapter 12 — Machine Utilities

What gets installed

Workspace scripts for inventory, storage benchmark, and GPU stress — no dedicated apt packages beyond shared GPU utils (`mesa-utils`, `vulkan-tools`).

Utility	Launcher	Script	Output
Dell inventory	`bin/dellmerge`	`scripts/dell/dellmerge.sh`	`dell-inventory/*.report` files
Storage survey	`bin/iotest`	`scripts/storage/iotest.sh`	I/O metrics (sudo)

Utility	Launcher	Script	Output
GPU stress	`bin/gpu-stress`	`scripts/gpu/gpu-stress.sh`	Vulkan/EGL per GPU
EFI / BIOS timing	`bin/efi-timing-suite`	`scripts/efi/efi-timing-suite.sh`	`B1GMB42.timing` (sudo)

Example reports in workspace: `b1gmb42.report`, `B1GMB42.ioperf`, `B1GMB42.timing` (from prior runs).

How it is installed

Scripts ship with the workspace. Rebuild Phase 10 runs `chmod +x` on `bin/*` and `scripts/*/*.sh`.

```
bin/dellmerge > b1gmb42.report
sudo bin/iotest
bin/gpu-stress 60 vkcube
sudo bin/efi-timing-suite          # before/after BIOS A/B changes
```

How to verify

```
[[ -x bin/dellmerge && -x bin/iotest && -x bin/gpu-stress && -x bin/efi-timing-su
bin/rebuild-machine --verify-only   # checks dellmerge, gpu-stress, iotest, apply
head -20 b1gmb42.report 2>/dev/null || bin/dellmerge | head -20
```

How to customize

- Edit `scripts/dell/dellmerge.sh` to add inventory fields
- `gpu-stress` accepts duration and backend (vkcube default)
- `iotest` targets block devices — read script header before running on production pools
- `efi-timing-suite` writes a machine-local timing baseline; re-run after BIOS changes for A/B compare

What rebuild does / does not do

Does	Does not
chmod utility launchers	Run dellmerge, iotest, or efi-timing automatically
Verify launcher executables exist	Archive reports to a fixed path

Chapter 13 — FactoryDocs (Workspace Archive)

What gets installed

Nothing on the Linux host automatically. FactoryDocs is a sorted archive of Dell T5810 vendor support packages for Windows recovery, firmware, and reference — stored only in the workspace.

Metric	Pre-crash	Current
Sorted packages	101	**19**
GPU drivers (FirePro/Quadro)	Yes	**Missing**
PERC H710 driver/firmware	Yes	**Missing**
Audio / input drivers	Yes	**Missing**
Win7/10/WinPE CAB packs	Yes	**Missing**

Full pre-crash file list: `FactoryDocs/MANIFEST-pre-crash.txt` (91 items flagged `MISS`).

Layout

Folder	Contents
`System-T5810/`	BIOS, chipset, ME, TPM, manuals
`GPU/`	AMD FirePro, NVIDIA (**empty — re-download**)
`Storage/`	PERC H710, Intel RST, SSD/HDD firmware
`Network/`	Intel Ethernet
`Dell-Management/`	Command Update, Configure
`Expansion-Cards/`	Serial, Thunderbolt docs
`_Misc/`	Non-Dell packages
`_incoming/`	Drop new Dell downloads here

How it is installed

Ingest new downloads:

```
cp ~/Downloads/* ~/Documents/IndianaDell/FactoryDocs/_incoming/
python3 ~/Documents/IndianaDell/FactoryDocs/_sort_factory_docs.py
```

Priority re-downloads from Dell T5810 drivers:

1. `GPU/AMD-FirePro/Windows/` — Video_Driver_C5FPW (W5000/W5100)
2. `Storage/RAID-Controller-PERC/Windows/` — PERC H710
3. `System-T5810/Windows-10/` — T5810-win10 CAB
4. `Audio/Windows/` — Audio_Driver_5P33P
5. `Dell-Management/Windows/` — Command Configure, Monitor

Windows installs use Dell CAB/EXE packages from these folders — not apt.

How to verify

```
find FactoryDocs -type f ! -path '*/_incoming/*' | wc -l
cat FactoryDocs/README.md
grep MISS FactoryDocs/MANIFEST-pre-crash.txt | wc -l
```

How to customize

- `_sort_factory_docs.py` dedupes and sorts by hardware category
- Cross-reference hardware manual: `B1GMB42-slot-port-inventory.md` for what hardware needs drivers

What rebuild does / does not do

Does	Does not
Nothing	Copy FactoryDocs to system paths Install Windows drivers Download missing CABs

FactoryDocs recovery is a **manual** ongoing task (Chapter 3 item 7).

Chapter 14 — Gaps and Limits

Documented boundaries of what IndianaDell does **not** install or support on this host.

Not installed

Item	Notes
SDRangel, SigDigger	Not on Flathub; use gqrx + URH + inspectrum
Rust SDR crate workspace	Toolchain only — add crates per project with `cargo add`
ZFS / disk layout tools	Out of scope — handled at OS install time
Windows / dual-boot	FactoryDocs holds CABs; no auto-install

Item	Notes
HackRF hardware test	No device attached at last verify (2026-07-05)
Ventoy full seed on every boot	Manual — run `~/bin/seed-ventoy-persistence.sh` after changes
PNY rebuild-stick seed automation	Manual loop-mount of `ubuntu-26.04.dat` (see Ch. 15); stick units only fix groups/SSH/hostname
Freerouting / pcb2gcode	Not installed — tscircuit ships `@tscircuit/capacity-autorouter`
`kicad-doc-id` 10.x	Universe still 9.0.8; omitted from `APT_KICAD`

Lost in TPM/ZFS crash

Item	Recovery
Pre-crash full apt list	**Not found** — use `apt-full-manifest.txt` from current rebuild
Pre-crash FactoryDocs	82/101 packages missing — see `MANIFEST-pre-crash.txt`
Pre-crash apt-hamradio list	Regenerated by rebuild `save_manifests()`

GPU / ROCm matrix

Capability	W5000 / W5100 on Ubuntu 26.04
Desktop amdgpu	Yes — with `etc/` configs
Vulkan (`vkcube`)	Yes
OpenCL (`clinfo`)	Yes (limited)
ROCm HIP / ML training	**No** — not in AMD ROCm support matrix for FirePro W5000/W5100

Use `bin/amd-preflight` before `bin/amd-install`; expect warnings for these GPUs.

Rebuild intentional omissions

These remain **manual** by design (see Chapter 3):

- `sudo bin/apply-amdgpu`

- `sudo bin/apply-zfs-smb` then `sudo smbpasswd -a user` (Samba /home share; Ch. 17)
- `bin/apply-dark-mode`, `bin/apply-max-performance`, `bin/fix-nautilus-desktop-launch`, `bin/sync-desktop-icons`
- `bin/themes-extract`, `sudo bin/themes-install-boot`
- `bin/hackrf-flash-mayhem` (DFU with hardware)
- `bin/amd-install` (optional ROCm)

Future enhancement could fold GNOME/theme steps into rebuild; current manual documents the gap explicitly.

Encryption / TPM

Hardware manual recommends **not** re-enabling ZFS encryption until TPM + recovery strategy is documented. IndianaDell rebuild does not touch encryption.

ZFS boot import

The installed host **must** keep force import enabled:

```
# /etc/default/zfs
ZPOOL_IMPORT_OPTS="-f"
```

`bin/rebuild-machine` does **not** write this file. After reinstall or recovery, verify it yourself (Chapter 3 checklist). Without `-f`, boot can hang when pools need force-import (post-export hostid mismatch, unclean shutdown).

When something fails

1. Read `scripts/rebuild/last-run.log`
2. Run `bin/rebuild-machine --verify-only` for targeted MISS lines
3. Re-run individual phases (apt, HackRF build, Mayhem download) manually
4. Check chapter-specific verify sections

Reporting issues

Capture:

```
bin/dellmerge > debug.report
bin/rebuild-machine --verify-only 2>&1 | tee verify.log
uname -a && lsb_release -a
```

Chapter 15 — Ventoy Live Session & Persistence

This chapter covers **two** Ventoy volumes used on the lab:

Volume	Friendly name	Role	Typical size
Internal Seagate ST500DM002	**Uncle Wiggly** □□	Full rabbit hole: many ISOs, 24 GB Ubuntu persistence, DOS-BOOT / Windows / ISO-STASH	~466 GB disk
PNY USB 2.0 (~30 GB)	**Rebuild stick**	Lean Ubuntu 26.04 live + **3 GB** persistence for recovery / thumper rebuild	~30 GB USB

Both use Ventoy: drop ISOs onto the data partition; they appear in the boot menu. Writable **casper-rw** persistence keeps login state across live boots.

EFI note (Tower5810): Uncle Wiggly's SATA port may be **disabled in Setup** for fast POST (`docs/fast-boot.md`). If `/dev/disk/by-label/Wiggly` is missing but the Seagate is cabled, re-enable that SATA port in **F2 Setup**, then reboot.

Overlay layout (both volumes)

Ubuntu live + Ventoy persistence uses an ext4 file labeled **`casper-rw`**:

```
persistence/ubuntu-26.04.dat   # ext4, LABEL=casper-rw
  upper/                       # modern overlay upper (preferred)
    home/<liveuser>/
    etc/
    usr/local/sbin/
    …
  work/                        # created by live system as needed
```

Older seeds may have used `cow/upper/`; current IndianaDell seeds use **`upper/`**. Seed scripts prefer `upper/` and remove a stale `cow/` tree when seeding.

Ventoy config (on the same data partition as the ISO):

```
{
    "persistence": [
        {
            "image": "/ubuntu-26.04-desktop-amd64.iso",
            "backend": "/persistence/ubuntu-26.04.dat",
            "autosel": 1
        }
    ]
}
```

Canonical repo copy for Uncle Wiggly: `scripts/ventoy/ventoy.json`. Paths are **absolute from the Ventoy data partition root**. Persistence backend and ISO must live on the **same** Ventoy data partition.

Subdirectories and non-ISO files

Ventoy **recursively scans** the data partition for bootable images (`.iso`, etc.). Subfolders are fine and already used on Wiggly:

Path example	Notes
`/perc/*.iso`	PERC FreeDOS/Linux kit; FreeDOS can use `auto_memdisk` in `ventoy.json`
`/Windows NT ISOs/*.iso`	Period OS media
`/QubesOS/Qubes-….iso` + `.asc` + signing key	Signatures sit next to the ISO; Ventoy ignores non-bootable files

Optional: empty `.ventoyignore` in a folder hides that tree from the boot menu.

Create a persistence image (example 3 GiB):

```
# From extracted Ventoy release, or scripts/ventoy/ if present
sudo ./CreatePersistentImg.sh -s 3072 -t ext4 -l casper-rw -o persistence/ubuntu-2
# Extend later:
sudo scripts/ventoy/ExtendPersistentImg.sh /path/to/ubuntu-26.04.dat <extra-MB>
# then resize2fs on the loop device if needed
```

Uncle Wiggly □□ (internal Ventoy)

Names: friendly **Uncle Wiggly**; partition label **`Wiggly`** (historically `sdc1`, mount `/mnt/wiggly`). Same Seagate also holds **DOSBOOT**, Windows, **ISO-STASH** — see hardware inventory.

What gets persisted (full seed)

Item	Live path	In casper image
User home	`/home/ubuntu` (Wiggly full seed convention)	`upper/home/ubuntu/`
Installed packages	dpkg overlay	`upper/var/lib/dpkg/`
GDM autologin	`/etc/gdm3/custom.conf`	`upper/etc/gdm3/`
Grok auth + sessions	`~/.grok/`	same (**never in git**)
GitHub CLI auth	`~/.config/gh/`	same
SSH keys	`~/.ssh/`	same
Chrome (tier C)	`~/.config/google-chrome/` curated	bookmarks, prefs, logins, Web Data, Extensions — **no caches**
Runtime secret source	`/home/user/` when ZFS rpool is available	pulled via `resolve-secrets.sh`
IndianaDell workspace	`~/Documents/IndianaDell/`	full tree (git clone or rsync)
PATH overrides	`~/.config/indianadell/path.sh`	same

Persistence image: `/persistence/ubuntu-26.04.dat` (**24 GB** ext4, label `casper-rw`) on label **`Wiggly`**.

One-time setup (Tower5810)

```
sudo mount -o uid=$(id -u),gid=$(id -g) /dev/disk/by-label/Wiggly /mnt/wiggly
bin/setup-wiggly-ventoy    # verify ISO, ventoy.json, .dat filesystem
```

Seed session state

```
~/bin/seed-ventoy-persistence.sh
# or:
PERSIST_MOUNT=/mnt/persist-check ~/bin/seed-ventoy-persistence.sh
SEED_CHROME=c ~/bin/seed-ventoy-persistence.sh   # default chrome tier
SEED_CHROME=off ~/bin/seed-ventoy-persistence.sh
```

Mode	When	Network?
Live casper overlay	Already booted from Ventoy persistence	**No** — local rsync only
External `.dat` seed	Seeding from Tower5810 / mounted volume	Only if IndianaDell must be **git cloned**

Network check: waits up to `SEED_NETWORK_WAIT_SECS` (default **120s**). Skip: `SEED_SKIP_NETWORK_CHECK=1`.

Chrome profile seed (`SEED_CHROME`)

Default **c**. Prefer `/home/user/.config/google-chrome` when rpool home exists; never copies Cache / Code Cache / GPU* / Service Worker.

Tier	What is copied
`off` / `0`	Nothing
a	Bookmarks + Preferences
b	a + Local State + Secure Preferences
c	b + Login Data + Web Data + Extensions + Local Extension Settings
d	Reserved (same as c for now)

Login experience (Wiggly full seed)

1. **GDM autologin** — live user **ubuntu** (Wiggly seed convention; see rebuild stick for **user**)
2. **PATH** — IndianaDell `bin/` and `scripts/` via `~/.config/indianadell/path.sh`
3. **Grok autostart** — often disabled (`X-GNOME-Autostart-enabled=false`)

4. **Installer** — no autostart; Desktop Install icon when needed

`resolve-secrets.sh` materializes secrets from `/home/user` when rpool exists, else Ventoy `$HOME`.

PNY rebuild stick (lean USB)

Device: ~30 GB PNY USB (model often `USB 2.0 FD`). **Not** production storage — install/recovery only.

Why repartition / reinstall Ventoy

The old layout was ~6 GB Ventoy + 32 MB VTOYEFI + ~24 GB DOSBOOT. Ubuntu 26.04 desktop ISO is **~6.1 GB**, so the small Ventoy slice could not hold ISO + persistence. Rebuild (2026-07) used Ventoy **1.1.16** force-install so almost the whole stick is one data partition:

Partition	Size	Label	Role
…1	~29.9 G	Ventoy	ISO + persistence + `ventoy.json`
…2	32 M	VTOYEFI	Ventoy EFI

```
# SAFETY: confirm USB transport + ~30 GB before -I
lsblk -o NAME,SIZE,MODEL,TRAN,LABEL
# From extracted ventoy-*-linux:
sudo bash ./Ventoy2Disk.sh -I -L Ventoy /dev/sdX   # double-confirm y/y
```

Hiren's / old Ubuntu 22 / DOSBOOT content on that stick was wiped by reinstall. BartPE and current media live on **Uncle Wiggly** (or ISO-STASH).

Layout on the stick

```
/ubuntu-26.04-desktop-amd64.iso          # ~6.1 G (copied from Wiggly)
/persistence/ubuntu-26.04.dat            # 3 G casper-rw
/ventoy/ventoy.json                      # persistence map, autosel=1
```

Capacity rule of thumb: 6.1 G ISO + 3 G `.dat` □ 9 G used; leave free space for future ISOs. A **24 GB** Wiggly-class overlay does **not** fit on this stick.

Lean seed policy (rebuild stick)

Include (as live user home — see identity below):

Item	Source (Tower5810)	Notes
SSH keys	`/home/user/.ssh/`	`id_rsa`, config, autho-rized_keys; mode 700/600
GitHub CLI	`/home/user/.config/gh/`	`hosts.yml` token
Grok	`/home/user/.grok/`	auth + sessions; **prune `downloads/`** to save space
GnuPG	`/home/user/.gnupg/`	optional
gitconfig	`/home/user/.gitconfig`	
Cursor auth	`~/.config/cursor/auth.json`	if present
Shell dots	`.bashrc`, `.profile`, …	
IndianaDell config	`~/.config/indianadell`, autostart, dconf	
`~/bin`	host `~/bin` helpers	
Keyrings	`~/.local/share/keyrings`	
Project PDFs only	`Documents/IndianaDell/B1GMB42-*.pdf`	manuals / trifolds / ZFS recovery — **not** full Documents tree

Exclude: full `Documents/`, `.cache`, Chrome bulk, cargo/rustup/wine/googleearth, full `.local` (except keyrings).

Seed path in the image: `upper/home/user/…` (after username change below).

Mount / edit / unmount pattern:

```
DAT=/run/media/$USER/Ventoy/persistence/ubuntu-26.04.dat   # or by-label Ventoy
MOUNT=/mnt/persist-usb
sudo mkdir -p "$MOUNT"
LOOP=$(sudo losetup --show -f "$DAT")
sudo mount "$LOOP" "$MOUNT"
# edit $MOUNT/upper/ ...
sync
sudo umount "$MOUNT"
sudo losetup -d "$LOOP"
```

Live user identity: `user` (not `ubuntu`)

Casper creates the live account from **`/etc/casper.conf`**. **`FLAVOUR` must be non-empty** or casper ignores `USERNAME` / `HOST` and substitutes the flavour string.

Seeded on the rebuild stick:

```
# /etc/casper.conf  (in upper/)
export USERNAME="user"
export USERFULLNAME="User"
export HOST="thumper"
export BUILD_SYSTEM="Ubuntu"
export FLAVOUR="Ubuntu"    # required for USERNAME/HOST to stick
```

Also:

File	Purpose
`upper/etc/gdm3/custom.conf`	`AutomaticLoginEnable=true`, `AutomaticLogin=user`
`upper/etc/sudoers.d/casper`	`user ALL=(ALL) NOPASSWD: ALL`
`upper/home/user/`	Seeded secrets + lean home (uid/gid **1000**)

Casper's `15autologin` / `25adduser` / `44pk_allow_ubuntu` all use `$USERNAME`, so PolicyKit and autologin follow `casper.conf` when FLAVOUR is set.

Boot unit: groups + rename safety net

Script: `upper/usr/local/sbin/indianadell-live-user.sh`
Unit: `indianadell-live-user.service` (WantedBy `multi-user.target` and `graphical.target`)
Log: `/var/log/indianadell-live-user.log`

Behavior:

1. If account **`ubuntu`** exists and **`user`** does not → `groupmod`/`usermod` rename + move home (old overlays / missed FLAVOUR).
2. Add **`user`** to every **special group that exists** on the live system (skip missing groups quietly).
3. Refresh sudoers + GDM autologin for `user`.
4. Set hostname **`thumper`** and `127.0.1.1 thumper.local thumper` in `/etc/hosts`.

Group list (lab / rebuild relevant — install packages later may create more groups; reboot re-runs attach):

Category	Groups
Admin / desktop	`adm`, `cdrom`, `sudo`, `dip`, `plugdev`, `lpadmin`, `sambashare`, `users`
Serial / radio	`dialout`, `tty`, `uucp`
AV / GPU / input	`audio`, `video`, `render`, `input`
Print / scan	`lp`, `scanner`
Network	`netdev`, `bluetooth`
Containers / VMs	`docker`, `lxd`, `kvm`, `libvirt`, `libvirt-qemu`
Other	`disk`, `floppy`, `ssl-cert`, `wireshark`, `fuse`

Matches the intent of Tower5810's host user groups (`id user`) plus hardware-facing groups we may install on live.

Fast-boot deferments (Tower5810 parity + live snap kill)

Same idea as `bin/apply-fast-boot` / `docs/fast-boot.md`, adapted for **casper overlay** (cannot run `systemctl disable` on the ISO lower layer — use **mask symlinks**).

Mechanism	Path in overlay	Role
Boot masks	`etc/systemd/system/<unit> → /dev/null`	Unit cannot start early (present as soon as root mounts)
Permanent mask list	`etc/indianadell-live-mask.list`	Never auto-start (snap seed, NM-wait-online, …)
Deferred list	`etc/indianadell-deferred.list`	Unmasked + `start --no-block` **after** `graphical.target`

Mechanism	Path in overlay	Role
Socket-lazy list	`etc/indianadell-socket-lazy.list`	`docker` / `cups` / `snapd` / `libvirt` sockets only
Early assert	`indianadell-live-fastboot.service` → `usr/local/sbin/indianadell-live-fastboot.sh`	Re-mask/stop thrashers; log `/var/log/indianadell-live-fastboot.log`
Post-desktop start	`indianadell-deferred.service` → `usr/local/sbin/indianadell-start-deferred`	After GDM; does **not** block desktop

Permanent masks (rebuild stick — the usual multi-minute hang):

- `snapd.seeded.service` (main culprit on Ubuntu live)
- `snapd.autoimport.service`, `snapd.core-fixup.service`, `snapd.recovery-chooser-trigger.service`, …
- `NetworkManager-wait-online.service`

Deferred (masked at boot, started later in background): `snapd.service`, cloud-init, *cups*, bluetooth, avahi, ModemManager, docker/libvirt stack, apport/whoopsie, unattended-upgrades, etc. (full list on the stick).

`snapd.service` itself is only started after the desktop is up (and only if you unmask/start it); **seeded** stays masked so it never blocks login.

Verify on live:

```
systemctl is-enabled snapd.seeded.service   # expect: masked
systemctl is-active snapd.seeded.service    # inactive
systemd-analyze blame | head -20
sudo tail /var/log/indianadell-live-fastboot.log
```

Plymouth theme (`indianadell`)

Tower's custom theme is seeded into the overlay (~20 MB):

Item	Overlay path
Theme tree	`usr/share/plymouth/themes/indianadell/` (from host `/usr/share/plymouth/themes/indianadell`)
Default alternative	`etc/alternatives/default.plymouth` → `…/indianadell.plymouth`
Default symlink	`usr/share/plymouth/themes/default.plymouth` → alternatives
Daemon config	`etc/plymouth/plymouthd.conf` → `Theme=indianadell`

`indianadell-live-fastboot.sh` re-asserts the default theme once rootfs is up.

Caveat: The **very first** splash frames still come from the **ISO initrd** (stock spinner/BGRT). After casper mounts the persistence overlay, Plymouth uses `Theme=indianadell` for late boot / shutdown / any restart of plymouthd. Fully replacing early ISO splash would require rebuilding `casper/initrd` on the ISO (not done on this stick).

Install source on Tower: `sudo bin/themes-install-boot` (see Ch. 5). Stick seed copies the **installed** theme tree, not a separate rebuild of animation frames.

Hostname `thumper` / `thumper.local` and OpenSSH on LAN

Hostname: `casper.conf` `HOST=thumper` plus `/etc/hostname` and `/etc/hosts` (`thumper.local`). Cloud-init can be steered with `upper/etc/cloud/cloud.cfg.d/99-thumper-hostname.cfg` if present.

SSH unit (optional seed): `thumper-lan-ssh.service` → `upper/usr/local/sbin/thumper-lan-ssh.sh`

- Waits for network if needed; installs **`openssh-server`** (apt, or debs under `/var/cache/thumper-debs/` if cached).
- Drop-in `/etc/ssh/sshd_config.d/99-thumper-lan.conf`: listen `0.0.0.0` / `::`, pubkey + password auth, no root login, no empty passwords.
- Enables/starts `ssh.service` (disables socket-only activation if needed).
- Allows UFW OpenSSH if UFW is active; starts `avahi-daemon` when available for mDNS **`thumper.local`**.
- Log: `/var/log/thumper-lan-ssh.log`

Login:

```
ssh user@thumper.local
# or LAN IP from the live session
```

Use the same SSH private key as Tower5810 (`id_rsa`); public key is in `~/.ssh/authorized_keys` on the stick. Live password is typically blank at the console (casper); SSH should use **keys** (empty passwords disabled for SSH).

MOTD hint (if seeded): `etc/update-motd.d/99-thumper-ssh`.

Secrets policy (`resolve-secrets.sh`)

Canonical secret relative paths (never commit to git):

```
.ssh
.grok
.config/gh
```

When ZFS rpool `/home/user` is present, it is the live secret **source**; Ventoy persistence is the portable **store** under the live home. Chrome seed is separate (`SEED_CHROME`).

ZFS recovery (rpool + bpool)

Manual: `docs/B1GMB42-zfs-recovery.md` + `B1GMB42-zfs-recovery.pdf` (repo root and DOSBOOT recovery kit).

Boot Ventoy Ubuntu live — **do not** use a broken installed system as root.

```
sudo apt-get install -y zfsutils-linux
cd ~/Documents/IndianaDell          # or recovery kit path
sudo ./mount-rpool-recovery.sh mount
sudo ./scripts/recovery/mount-bpool-recovery.sh mount
sudo ./mount-rpool-recovery.sh chroot
# repair; then exit and umount both scripts
```

Before rebooting installed OS: `/etc/default/zfs` must set `ZPOOL_IMPORT_OPTS="-f"`. Kernel one-shot: `zfsforce=1`.

Deploy kit: `bin/deploy-dosboot-recovery` (from Tower5810). PDF on the rebuild stick: `~/Documents/IndianaDell/B1GMB42-zfs-recovery.pdf`.

GitHub repository

https://github.com/webaugur/IndianaDell (private)

```
bin/pull-repo --verify
bin/pull-repo --dragonsdr
```

`bin/push-repo`

HTTPS optional: `INDIANADELL_REMOTE=…` after `gh auth login`. Large FactoryDocs use **Git LFS**.

On the rebuild stick, after network is up: `git clone git@github.com:webaugur/IndianaDell.git` using seeded SSH keys.

How to verify

Uncle Wiggly (full)

```
findmnt / | grep -qE 'cow|overlay' && echo "persistence overlay active"
grep AutomaticLogin= /etc/gdm3/custom.conf
echo "$INDIANADELL_ROOT"
which dellmerge pull-repo push-repo grok
bin/pull-repo --verify
```

PNY rebuild stick

```
# Identity
whoami                    # expect: user
id                        # expect sudo,adm,plugdev,dialout,… as available
hostname; hostname -f     # thumper / thumper.local
grep AutomaticLogin=user /etc/gdm3/custom.conf

# Persistence
findmnt / | grep -qE 'cow|overlay' && echo "overlay ok"
test -f ~/.ssh/id_rsa && test -f ~/.config/gh/hosts.yml && test -f ~/.grok/auth.js
ls ~/Documents/IndianaDell/*.pdf

# SSH on LAN (after thumper-lan-ssh has run)
ss -tlnp | grep ':22'
systemctl is-active ssh
# from another host:
#   ssh user@thumper.local

# Fast-boot / snapd not thrashing
systemctl is-enabled snapd.seeded.service   # masked
systemd-analyze blame | head -15

# Plymouth (rootfs theme)
grep Theme= /etc/plymouth/plymouthd.conf    # indianadell
readlink -f /etc/alternatives/default.plymouth

# Logs
```

```
sudo tail -50 /var/log/indianadell-live-user.log
sudo tail -50 /var/log/thumper-lan-ssh.log
sudo tail -50 /var/log/indianadell-live-fastboot.log
```

How to customize

Goal	Action
Re-seed Wiggly full session	`~/bin/seed-ventoy-persistence.sh`
Change Grok session	Edit `GROK_SESSION_ID` in `grok-indianadell-launch.sh`
Enlarge persistence	`ExtendPersistentImg.sh` + `resize2fs`
Verify Wiggly layout	`bin/setup-wiggly-ventoy`
Edit rebuild stick overlay	loop-mount `ubuntu-26.04.dat`, edit `upper/`, unmount
Live user name	`upper/etc/casper.conf` (USERNAME + **`FLAVOUR`**) + GDM + home dir + `indianadell-live-user.sh`
Extra groups	Edit `SPECIAL_GROUPS` in `indianadell-live-user.sh`
SSH / hostname	`thumper-lan-ssh.sh` + sshd drop-in + hosts/hostname
Boot defer / mask lists	`etc/indianadell-deferred.list`, `etc/indianadell-live-mask.list`
Refresh Plymouth on stick	rsync host `/usr/share/plymouth/themes/indianadell/` into overlay; keep `plymouthd.conf`
Hide ISO folder from menu	`.ventoyignore` in that folder
Qubes + signatures	e.g. `Wiggly/QubesOS/*.iso` + `.asc` + key (subdirs OK)

Related tools

Tool	Role
`bin/setup-wiggly-ventoy`	Verify Wiggly ISO + `ventoy.json` + `.dat`
`bin/boot-uncle-wiggly-vm`	QEMU live + persistence smoke test
`bin/boxes-import-wiggly-isos`	GNOME Boxes domain per ISO on Wiggly

Tool	Role
`bin/setup-perc-ventoy`	PERC FreeDOS/IT kit on Wiggly (`perc/`)
`~/bin/seed-ventoy-persistence.sh`	Full seed into casper image
`scripts/ventoy/resolve-secrets.sh`	Secret path policy + Chrome tiers
`scripts/ventoy/ventoy.json`	Canonical Wiggly Ventoy plugin config
Ventoy release `CreatePersistentImg.sh`	Build empty `casper-rw .dat`

What rebuild does / does not do

Does	Does not
Install Chrome, gh, git-lfs when run on a full live session	Auto-configure Ventoy `ventoy.json`
Document seed / stick layout in this chapter	Auto-run seed every boot (except stick units: groups/SSH) Manage Seagate partition map (Wiggly/DOSBOOT/Windows) Fit a 24 GB Wiggly `.dat` on the 30 GB PNY stick

Chapter 16 — QEMU

QEMU is the core emulation and virtualization engine used throughout the IndianaDell / DragonSDR lab. On Tower5810 it provides the full range of system emulation targets, KVM-accelerated guests, and the shared-library builds required by Velxio / picsimlab for ESP32 development.

The complete upstream documentation lives in the QEMU source tree under `docs/`. This chapter gives a high-level map of that documentation so you can quickly locate the relevant sections.

Sub-chapters (QEMU docs tree)

About

High-level information about QEMU itself:

- Supported build platforms and minimum requirements
- Emulation capabilities and architecture coverage
- Deprecated and removed features
- License and contribution overview

Devel

Developer and internals documentation (most relevant when working on QEMU itself or debugging deep issues):

- Build system, Kconfig, and module architecture
- TCG (Tiny Code Generator) internals and plugins
- QAPI/QOM code generation, memory model, RCU, atomics
- Block layer, migration, multi-threaded TCG, iothreads
- Secure coding practices, style guide, patch submission process
- Tracing, replay, and record/replay infrastructure

Interop

Interoperability specifications and external interfaces:

- QEMU Machine Protocol (QMP)
- Guest agent protocol
- Block replication, COLO (COarse-grained LOck-stepping) fault tolerance
- NVDIMM, memory hotplug, PCI expander bridges, SR-IOV, etc.

Specs

Hardware and firmware specifications that QEMU emulates or interacts with:

- ACPI, SMBIOS, device tree fragments
- Virtio, vhost, and paravirtualized device specifications
- Firmware and boot interface details

Building QEMU on Tower5810

The DragonSDR build script `tools/emulators/qemu-lcgamboa/build-all.sh` produces both:

- The special Velxio/ESP32-compatible shared libraries (`libqemu-xtensa.so`, `libqemu-riscv32.so`)
- Full-featured shared libraries for all other architectures with PipeWire, KVM, virglrenderer, Spice, vhost, and modern storage/networking support enabled by default.

See the script and its pinned commit for the exact feature set.

Further reading

For the absolute latest and most detailed information, always consult the `docs/` directory inside the QEMU source tree you are building. The structure described above is stable across recent QEMU releases.

Chapter 17 — ZFS SMB share (`/home`)

On Linux, OpenZFS `sharesmb` is **not** a kernel SMB server. It registers a Samba **usershare** (`net usershare`) and expects **`smbd`** to listen on TCP 445. Tower5810 shares the existing dataset `rpool/USERDATA/home_ulmkkv` (`mountpoint=/home`), which contains both `user` and `ubuntu`. `/root` is not shared.

What gets installed

Piece	Role
apt `samba`	`smbd` + `net` (required for `zfs share` SMB)
apt `smbclient`	local smoke tests
`etc/samba/indianadell.conf`	LAN bind, no guest, `[home]` alias only
`sharesmb=on`	`rpool/USERDATA/home_ulmkkv`
`sharesmb=off`	`rpool/USERDATA` (no inherit) and `rpool/USERDATA/root_ulmkkv`

Clients: `\\10.0.0.113\home`, `\\10.0.0.113\Documents` (`/home/user/Documents`), and the ZFS usershare `rpool_USERDATA_home_ulmkkv`.

iPad / iOS Files: needs `vfs objects = catia fruit streams_xattr`. Without fruit, Files.app lists the tree and opens directories RDONLY (see `smbstatus`) so writes to `user/Documents` fail even though `[home]` is `read only = no`.

Do not set `usershare template share = home`. Samba 4.23.6 `load_usershare_service` **SIGSEGV** on iPad TCON to the ZFS usershare (`/var/log/samba/log.mobile`). Template is the unavailable share `zfs-usershare-template` (`available = no`, `read only = no`). After a config change, delete the Files.app server and add it again (reboot alone can reuse a stale TCON).

How it is installed

After Samba is on disk (rebuild Phase 2 installs `samba` + `smbclient`):

```
sudo bin/apply-zfs-smb
sudo smbpasswd -a user    # SMB password is a separate database
```

`apply-zfs-smb` copies the drop-in, inserts `include = /etc/samba/indianadell.conf` into **`[global]`** of `/etc/samba/smb.conf` (not after `[print$]` — that would leave usershares disabled), sets the three `sharesmb` properties, restarts smbd, then `zfs share -a smb`.

Do **not** install Samba and skip `apply-zfs-smb`. Ubuntu's installer left `sharesmb=on` on `rpool/USERDATA`, which would inherit onto **`/root`**.

How to verify

```
zfs get -r sharesmb rpool/USERDATA
# home_ulmkkv on (local); USERDATA and root_ulmkkv off
sudo net usershare list
# rpool_USERDATA_home_ulmkkv
ss -lnt | grep ':445'
# 10.0.0.113 and 127.0.0.1 only — not docker0/virbr0, not global IPv6
smbclient -L //10.0.0.113 -U user
smbclient //10.0.0.113/home -U user -c 'ls'
smbclient //10.0.0.113/Documents -U user -c 'ls'
```

Anonymous tree connect to `[home]` must be `NT_STATUS_ACCESS_DENIED`.

How to customize

- LAN address: `interfaces` in `etc/samba/indianadell.conf` (today `10.0.0.113`). Re-run `sudo bin/apply-zfs-smb` after edits.
- Share name: `[home]` in that file. The ZFS usershare name follows the dataset.
- Narrower share (Documents only, or `/home/user` without ubuntu): smb.conf `path =`, not `sharesmb` — `sharesmb` can only export a **dataset**.
- UFW is inactive on this host. If you enable it, allow TCP 445 from `10.0.0.0/24` only.

What rebuild does / does not do

Does	Does not
apt-install `samba` and `smbclient` (APT_CORE)	Run `apply-zfs-smb`

Does	Does not
	Set `smbpasswd` Change ZFS `sharesmb` (survives reboot once set)

Post-rebuild: `sudo bin/apply-zfs-smb` then `sudo smbpasswd -a user`.

Chapter 18 — Disk IO tuning (UI responsiveness)

Symptom: the GNOME desktop freezes whenever the disks are saturated (builds, `dsd-fme` decoders, swap-out, ZFS txg flushes).

Cause (Tower5810, measured 2026-10-27):

Finding	Detail
Root pool on a spinning disk	`rpool` data vdev is the 1.4 TB Hitachi HDS723015BLA642 (`sdb4`, rotational).
SSD is not free space	The TEAM T253X6256G (`sda`, 238 GB) is a **ZFS `special` vdev** (metadata + blocks ≤ 32K). It cannot be carved into a non-ZFS swap partition without removing it from the pool.
Swap on a ZFS zvol	`rpool/swap` (`/dev/zd0`, 33 GB, `sync=always`, `volblocksize=8K`). Every swap-out is a synchronous HDD write. A second 4 GB swap partition sat on the same HDD (`sdb3`).
Unbounded ARC	`zfs_arc_max=0` → ARC allowed up to all 16 GB RAM, competing with apps and pushing them into that swap.
Pressure stall info	`/proc/pressure/io` full avg300 ≈ 10 % while CPU pressure was ≈ 2 %: the stalls were disk, not CPU.
IO scheduler	`mq-deadline` on all disks; no per-process fairness, so one writer (`cargo`, ~40 MB/s) starves the desktop.

Any IndianaDell machine: `bin/apply-tuning`

The same scripts now work on every similar-era Dell (Safar, Tower5810, the others), not only Tower5810. They detect the hardware and choose sizes from it. There is one entry point:

```
bin/apply-tuning report            # no root: detected hardware, live status, and
bin/apply-tuning status            # no root: each step ok / DIFFERS / skip
bin/apply-tuning apply --dry-run   # no root: exact actions, changes nothing
sudo bin/apply-tuning apply        # root steps
bin/apply-tuning apply             # user steps (chrome-cap, max-performance), no
sudo bin/apply-tuning revert       # undo the IO steps
# --only zram,sysctl   --skip fast-boot   --config FILE
```

Steps: `arc` `zram` `sysctl` `diskswap` `bfq` `initramfs` `ionice` `fast-boot` `fast-login` (root) and `chrome-cap` `max-performance` (user). Running a root step without sudo prints the exact `sudo` command instead. Every step is idempotent: a file is rewritten only when its content differs.

What is detected	How it is used
RAM (`MemTotal`)	zram = **75 % of RAM, rounded to GiB, clamped 2G..32G** (16 GB → 12G, 8 GB → 6G, 32 GB → 24G). ZFS ARC = **25 % of RAM, clamped 1..16 GiB**. Chrome MemoryMax = 37.5 % of RAM (min 2G), MemoryHigh = max − 1G.
ZFS module loaded	ARC cap and the initramfs rebuild only when ZFS is present; otherwise skipped. Swap zvols are noted, never destroyed.
Root fs, initramfs tool	`update-initramfs -u` or `dracut -f`, only when the ARC file changed.
Disks (`lsblk` ROTA/TRAN)	BFQ for rotational SATA/SAS disks only. SSD, NVMe and eMMC keep their default. USB disks are left alone (flash sticks report rotational=1).
Disk-backed swap	`swapoff` only if its used pages fit in MemAvailable − 1.5 GB. Active fstab lines whose type field is swap are commented `#disabled-by-io-tuning#`, after a timestamped backup in `/root/io-tuning-backup/`.

What is detected	How it is used
Other zram managers	zram-generator, zram-config or zramswap: our zram step is skipped (no double zram) unless `ZRAM_TAKEOVER=1`.
GRUB, NetworkManager, kdump	fast-boot skips what is missing. The cmdline is built from the machine's own `GRUB_CMDLINE_LINUX_DEFAULT` with `crashkernel=` removed and `quiet splash` kept, so options such as Tower5810's `usbcore.quirks=` survive.
Display manager, session	fast-login (GDM auto-login) is **opt-in** (`ENABLE_FAST_LOGIN=1`) and GDM-only. max-performance needs GNOME.
Laptop (DMI chassis or battery)	max-performance is skipped unless `FORCE_DESKTOP_POWER=1`.

Configuration (later wins): computed defaults < `etc/indianadell/hosts/<hostname>.conf` in the repo < `/etc/indianadell/tuning.conf` < `--config FILE` < environment variables. The keys are listed in `etc/indianadell/tuning.conf.example`. `etc/indianadell/hosts/Tower5810.conf` pins Tower5810 to what was tuned there: 13G zram, 4 GiB ARC, Chrome 5G/6G, and auto-login for `user`. A new machine needs no file. To pin it, copy the example to `/etc/indianadell/tuning.conf`, or add `hosts/<name>.conf` to the repo.

New machine (e.g. Safar)

```
cd ~/Documents/IndianaDell && git pull
bin/apply-tuning report > /tmp/tuning-$(hostname).txt; cat /tmp/tuning-$(hostname
sudo bin/apply-tuning apply            # root steps
bin/apply-tuning apply                 # Chrome cap + GNOME max performance (as yo
bin/apply-tuning status                # everything ok; reboot, then systemd-analy
```

The older launchers still work and call the same code. `sudo bin/apply-io-tuning` runs the IO steps only, and `bin/apply-io-tuning --report` reports them. `sudo bin/resize-zram-swap` without a size uses the configured size. `bin/apply-fast-boot --dry-run` needs no sudo.

What `apply-io-tuning` changes (Tower5810 history)

```
sudo bin/apply-io-tuning
# options (environment): ARC_MAX_GIB=4  ZRAM_SIZE=13G  BG_PROCS="cargo rustc dsd-
```

1. **ZFS ARC cap** — 4 GiB live (`/sys/module/zfs/parameters/zfs_arc_max`) and persistent in `/etc/modprobe.d/zfs-arc.conf`.
2. **zram swap** — `zram-swap.service`: 13 GiB (was 8 GiB; resize live with `sudo bin/resize-zram-swap 13G`), zstd, priority 100. Swap lives in compressed RAM, zero disk IO.
3. **sysctl** — `/etc/sysctl.d/60-zram.conf`: `vm.swappiness=100`, `vm.page-cluster=0` (the right values when swap is zram; high swappiness is cheap there).
4. **Disk swap off** — each non-zram entry in `/proc/swaps` is `swapoff`'d only if its used pages fit in `MemAvailable` minus 1.5 GB; matching `/etc/fstab` lines are commented with `#disabled-by-io-tuning#`. Original fstab is saved at `/root/io-tuning-backup/fstab.orig`.
5. **BFQ scheduler** — `modprobe bfq`, loaded at boot, plus `/etc/udev/rules.d/60-hdd-bfq.rules` for every rotational `sd*` disk (USB disks are skipped by the live step). BFQ gives interactive/latency-sensitive IO priority over bulk writers.
6. **initramfs** — `update-initramfs -u` so the ARC cap applies from boot (ZFS root).
7. **Idle IO class** for running `cargo`, `rustc`, `dsd-fme`, `commander`.

The `rpool/swap` zvol is **left in place**. Once you are happy, reclaim the space with `sudo zfs destroy rpool/swap`.

Running heavy jobs without freezing the desktop

```
bin/run-in-background cargo build --release     # ionice -c3 + nice -n 19
```

`ionice -c3` (idle class) is honoured by BFQ; it is **not** enforced under `mq-deadline`, which is why step 5 matters.

Verify

```
swapon --show                     # only /dev/zramN
cat /sys/block/sdb/queue/scheduler  # [bfq]
grep -E '^(size|c_max) ' /proc/spl/kstat/zfs/arcstats
cat /proc/pressure/io             # watch 'full' under load
```

Undo

```
sudo bin/revert-io-tuning
```

Removes the zram unit, ARC cap, sysctl, BFQ rule, restores fstab, and returns the disks to `mq-deadline`.

Notes and limits

- `sudo` is **not available** to the Grok Bot assistant on Tower5810 (its shell runs with `no_new_privs`). Run these scripts yourself from a terminal.
- Device names (`sdb`, `sdc`) are not hard-coded: BFQ is applied to every rotational `sd*` disk.
- The real fix for the root cause is more free SSD: moving `rpool`'s data vdev to SSD, or adding a mirror/L2ARC/SLOG on a non-special SSD, would help more than any tuning. Not done here.
- Swapping out a large zvol swap needs free RAM; if step 4 reports "not enough free RAM", close apps and re-run or reboot.

Resizing the zram swap

zram uses RAM only for pages actually stored (about 3.5:1 compression on this machine), so a larger device costs nothing until it fills. To change the size live and persistently:

```
sudo bin/resize-zram-swap 13G      # or 16G; edits zram-swap.service, then swapof
swapon --show                      # confirm the new size
zramctl --bytes                    # DATA = stored, TOTAL = RAM actually used
```

The script refuses to swap off if the stored pages would not fit in free RAM; it still saves the size for the next boot.

Capping Chrome's memory

Chrome was one of the largest RAM users on 16 GB. Spoofing the RAM size is not practical, so Chrome runs inside a kernel memory budget instead:

```
bin/apply-chrome-cap                 # no sudo; defaults MemoryHigh=5G MemoryMax=6
bin/apply-chrome-cap --high 4G --max 5G
bin/apply-chrome-cap --revert
```

- `~/.config/systemd/user/chrome.slice` holds the budget (also `CPUWeight=80`, `IOWeight=50`). All Chrome windows and PWAs share it.
- Past `MemoryHigh` the kernel reclaims Chrome's pages into zram swap (Chrome slows, the desktop does not). At `MemoryMax` only Chrome is OOM-killed.
- `~/bin/chrome-capped` (from `scripts/gnome/chrome-capped.sh`) starts Chrome with `systemd-run --user --scope --slice=chrome.slice` plus `--renderer-process-limit=8` and `--js-flags=--max-old-space-size=3072`. Extra flags: `CHROME_CAPPED_FLAGS`.
- The installer points the system launchers (copied to `~/.local/share/applications`) and the Chrome-made PWA launchers at the wrapper. Originals are in `~/Backups/chrome-cap`. Chrome

rewrites its PWA launchers on update, so re-run `bin/apply-chrome-cap` afterward.

- Terminal: the installer writes a marker-delimited block (`# >>> indianadell chrome-cap >>>`) to `~/.bash_aliases` that aliases `google-chrome`, `google-chrome-stable` and `chrome` to the wrapper. Re-running replaces the block; `--revert` removes it. Open a new terminal (or `source ~/.bash_aliases`) to pick it up.
- Also useful, in Chrome: Settings Performance Memory Saver.

What the cap covers (and doesn't)

Chrome moves its own main browser process (about 250-300 MB) into a separate per-app group that GNOME creates (`app-com.google.Chrome-<pid>.scope`), so that process is **not** inside `chrome.slice`. This is deliberate and left alone: the renderers, GPU, network and extension helper processes are the ones that balloon, and they all start inside the capped group and share its MemoryHigh/MemoryMax budget. Capping the main process too would risk the kernel killing the whole browser instead of only its helpers.

Check where each process lives:

```
for p in $(pgrep -f /opt/google/chrome/chrome); do echo "$(cut -d/ -f6- /proc/$p/c
systemctl --user show chrome.slice -p MemoryCurrent -p MemoryHigh -p MemoryMax
```

`MemoryCurrent` therefore reads lower than Chrome's own task manager (Shift+Esc) by roughly the main process's size.

Automatic re-check after updates

`bin/apply-chrome-cap` also installs three user units (no sudo): `chrome-cap-check.timer` (3 minutes after login, then hourly; it catches the nightly `apt-daily-upgrade`, which runs around 06:00), `chrome-cap-check.path` (fires when Chrome rewrites a launcher in `~/.local/share/applications`), and `chrome-cap-check.service`, which runs `scripts/gnome/check-chrome-cap.sh`. The script does nothing when every Chrome launcher still starts the wrapper. If an update reset one, or the wrapper or slice file is gone, it re-runs `bin/apply-chrome-cap` with your existing limits and sends a desktop notification.

```
systemctl --user list-timers chrome-cap-check.timer    # next run
journalctl --user -u chrome-cap-check -n 20            # history: "intact" or "re
systemctl --user start chrome-cap-check.service        # run the check now
```

`bin/apply-chrome-cap --revert` removes the units as well. The check re-applies launchers only; a Chrome window already running keeps its old cgroup until you restart it.

Repeat from scratch (one block)

Everything in this chapter is scripted and idempotent. On a fresh or rebuilt Tower5810, from `~/Documents/IndianaDell`:

```
# 1. Root-level IO tuning: zram swap, ZFS ARC cap, BFQ, idle IO for background jo
sudo bin/apply-io-tuning
sudo swapoff /dev/zd0 2>/dev/null || true      # finish emptying the old ZFS swap
# 2. User-level, no sudo: capped Chrome + launchers + terminal aliases
bin/apply-chrome-cap                           # --high 5G --max 6G are the defau
source ~/.bash_aliases
# 3. Make the new commands available in $PATH
bin/install-to-bin
# 4. Verify
swapon --show                                  # only /dev/zramN
cat /sys/block/sdb/queue/scheduler             # [bfq]
systemctl --user show chrome.slice -p MemoryHigh -p MemoryMax
type chrome                                    # aliased to ~/bin/chrome-capped
```

Undo: `sudo bin/revert-io-tuning` and `bin/apply-chrome-cap --revert`.

Appendix A — bin/ Launchers

All launchers live in `~/Documents/IndianaDell/bin/`. **PATH** is set automatically via `~/.config/indianadell/path.sh` (IndianaDell tools override system binaries).

Launcher	Runs	Chapter
`rebuild-machine`	`scripts/rebuild/rebuild-machine.sh`	2
`build-software-manual`	`scripts/docs/build-software-manual.sh`	1
`build-all-docs`	`scripts/docs/build-all-docs.sh`	1, 3
`pull-repo`	`scripts/github/pull-all.sh` — IndianaDell + LFS (`--dragonsdr` optional)	15
`install-dragonsdr`	`~/Documents/DragonSDR/bin/install-suite` — full SDR suite	8, 15

Launcher	Runs	Chapter
`push-repo`	`bin/push-repo` → GitHub `webau-gur/IndianaDell` (SSH default)	15
`setup-wiggly-ventoy`	`scripts/ventoy/setup-wiggly-ventoy.sh` — Uncle Wiggly ← ISO + ventoy.json + .dat	15
`setup-perc-ventoy`	`scripts/perc/setup-perc-ventoy.sh` — H710 FreeDOS/IT kit on Uncle Wiggly	hardware / PERC doc
`boot-uncle-wiggly-vm`	`scripts/ventoy/boot-uncle-wiggly-vm.sh` — QEMU live+persistence test	15
`boxes-import-wiggly-isos`	`scripts/ventoy/boxes-import-wiggly-isos.sh` — Boxes VM per ISO on Wiggly	15
`themes-preview-boot`	`Themes/scripts/plymouth-preview.py` — safe Plymouth window	5
`apply-fast-login`	`scripts/gnome/apply-fast-login.sh` — GRUB 0s + GDM autologin + face	5, 7
`apply-fast-boot`	`scripts/gnome/apply-fast-boot.sh` — strip crashkernel, defer daemons	5, 7 — `docs/fast-boot.md`

Launcher	Runs	Chapter
`build-zfs-recovery-doc`	`scripts/docs/build-zfs-recovery-doc.sh`	2, 15
`build-trifold-slick`	`docs/sales/B1GMB42-trifold.html` → sales PDFs	2
`deploy-dosboot-recovery`	`scripts/recovery/deploy-to-dosboot.sh`	2, 15
`efi-timing-suite`	`scripts/efi/efi-timing-suite.sh`	6, 12
`dellmerge`	`scripts/dell/dellmerge.sh`	12
`gpu-stress`	`scripts/gpu/gpu-stress.sh`	6, 12
`iotest`	`scripts/storage/iotest.sh`	12
`apply-tuning`	`scripts/tuning/indianadell-tuning.sh` — hardware-adaptive tuning for any IndianaDell Dell: `report` / `status` (no sudo), `apply [--dry-run] [--only/--skip]`, `revert`	18
`apply-io-tuning`	`scripts/storage/apply-io-tuning.sh` — zram swap, ARC cap, BFQ (run with sudo)	18
`revert-io-tuning`	`scripts/storage/revert-io-tuning.sh` — undo the above	18
`apply-chrome-cap`	`scripts/gnome/apply-chrome-cap.sh` — memory-capped Chrome slice + launcher wrapper	18

Launcher	Runs	Chapter
`resize-zram-swap`	`scripts/storage/resize-zram-swap.sh` — resize the zram swap live (`sudo bin/resize-zram-swap 13G`)	18
`run-in-background`	`scripts/storage/run-in-background.sh` — `ionice -c3 nice -n 19 <cmd>`	18
`apply-amdgpu`	`etc/apply.sh`	6
`apply-zfs-smb`	`scripts/samba/apply-zfs-smb.sh` — Samba + `sharesmb` for `/home`	17
`apply-sensor-watch`	`bin/apply-sensor-watch` — install thermal/fan watchdog unit	6
`indiana-sensor-watch`	`scripts/sensors/indiana-sensor-watch.sh` — `--once` / daemon	6
`indiana-monitor-input`	`scripts/display/indiana-monitor-input.sh` — DDC/CI input select	6
`indiana-ir-send`	`scripts/ir/indiana-ir-send.sh` — USB serial to Uno IR blaster	6
`indiana-ir-flash`	`scripts/ir/indiana-ir-flash.sh` — arduino-cli compile/upload	6

Launcher	Runs	Chapter
`samsungtv`	pipx `samsungtvws[cli]` — Tizen LAN remote (defaults to `10.0.0.31:8002`)	6
`tsci` / `tscircuit`	tscircuit CLI under `~/.local` — TypeScript → KiCad export	EDA / Ch. 3
`amd-install`	`amd-radeon/install-all.sh`	6
`amd-preflight`	`amd-radeon/00-preflight.sh`	6
`amd-verify`	`amd-radeon/04-verify.sh`	6
`amd-uninstall`	`amd-radeon/uninstall.sh`	6
`apply-dark-mode`	`scripts/gnome/apply-dark-mode.sh`	5, 7
`apply-max-performance`	`scripts/gnome/apply-max-performance.sh`	7
`fix-nautilus-desktop-launch`	`scripts/gnome/fix-nautilus-desktop-launch.sh`	3, 7
`sync-desktop-icons`	`scripts/gnome/sync-desktop-icons.sh`	3, 7
`themes-extract`	`Themes/scripts/extract-all.sh`	5
`themes-install-boot`	`Themes/scripts/install-boot-theme.sh`	5
`themes-restore-boot`	`Themes/scripts/install-boot-theme.sh --restore-stock`	5
`hackrf-env`	sources `DragonSDR/hackrf/scripts/env.sh`	10

Launcher	Runs	Chapter
`urh`	`DragonSDR/bin/urh`	10
`hackrf-setup-udev`	`DragonSDR/bin/hackrf-setup-udev`	10
`hackrf-download-mayhem`	`DragonSDR/bin/hackrf-download-mayhem`	10
`hackrf-prepare-sdcard`	`DragonSDR/bin/hackrf-prepare-sdcard`	10
`hackrf-flash-mayhem`	`DragonSDR/bin/hackrf-flash-mayhem`	10
`hackrf-build-mayhem`	`DragonSDR/bin/hackrf-build-mayhem`	10
`webm-to-mp4`	`scripts/media/webm-to-mp4.sh` — VP8 WebM → H.264 MP4	—
`clip-video`	`scripts/media/clip-video.sh` — trim `--video-start`/`--video-end` (`--start`/`--end`) to `.mp4` (H.264/AAC) or `.webm` (VP9/Opus); `--audio` replaces the clip audio	—

Ventoy session (`scripts/ventoy/` → `~/bin` via `install-ventoy-session.sh`):

Script	Purpose
`seed-ventoy-persistence.sh`	Snapshot session into Ventoy casper image (Uncle Wiggly full seed)
`seed-network-check.sh`	Internet/DNS check before seed
`grok-indianadell-launch.sh`	Seed then Grok fullscreen autostart
`install-ventoy-session.sh`	Install helpers, autostart, PATH
`mount-rpool-recovery.sh`	ZFS rpool chroot recovery (workspace root)

Script	Purpose
`mount-bpool-recovery.sh`	ZFS bpool mount at `/recovery/boot` (`scripts/recovery/`)

PNY rebuild stick (lean 3 GB persistence, live user `user`, groups, `thumper`/SSH): no dedicated `bin/` launcher yet — procedure and overlay paths are documented in **Chapter 15** (`upper/usr/local/sbin/indianadell-live-user.sh`, `thumper-lan-ssh.sh`).

Note: `hackrf-env` must be **sourced**, not executed: `source bin/hackrf-env`

Sudo required: `apply-amdgpu`, `themes-install-boot`, `themes-restore-boot`, `iotest`, `apply-tuning apply` (root steps), `apply-io-tuning`, `revert-io-tuning`, `resize-zram-swap`, `hackrf-setup-udev` (udev install), `amd-install`, `efi-timing-suite`, `mount-rpool-recovery.sh`, `mount-bpool-recovery.sh`

Appendix B — Apt Packages by Chapter

Workstation packages: `scripts/rebuild/package-lists.sh` (`APT_CORE` + `APT_KICAD`).
SDR / ham / HackRF packages: `~/Documents/DragonSDR/tools/package-lists.sh` (`APT_SDR`, `APT_HAM`, `APT_SDR_BUILD`).
Install SDR suite: `bin/install-dragonsdr` → DragonSDR `bin/install-suite`.
Full system snapshot: `apt-full-manifest.txt` (after rebuild).
SDR/ham filter snapshot: `apt-hamradio-dev-manifest.txt`.
KiCad opt-out: `SKIP_KICAD=1` (skips PPA + `APT_KICAD` + tscircuit npm on rebuild and `fix-indianadell`).
samsungtv opt-out: `SKIP_SAMSUNGTV=1` (skips pipx `samsungtvws[cli]` on rebuild and `fix-indianadell`).

Chapter 4 — Development (IndianaDell `APT_CORE`)

`build-essential`, `cmake`, `pkg-config`, `git`, `curl`, `wget`, `unzip`, `python3-pip`, `python3-venv`, `python3-dev`, `python3-numpy`, `python3-scipy`, `python3-matplotlib`, `python3-yaml`, `python3-requests`, `python3-pyqt5`, `python3-psutil`, `libssl-dev`, `clang`, `llvm-dev`, `libclang-dev`, `libusb-1.0-0-dev`, `libfftw3-dev`, `libvolk-dev`, `portaudio19-dev`, `libsndfile1-dev`, `libboost-dev`, `libboost-program-options-dev`, `pandoc`, `texlive-latex-recommended`, `texlive-fonts-recommended`, `texlive-xetex`, `gh`, `samba`, `smbclient`

Chapter 6 — GPU and Display

vulkan-tools, mesa-utils, mesa-utils-bin, clinfo, x11-apps, smartmontools, ddcutil, arduino-cli, ffmpeg

samsungtv (not apt; default-on): scripts/rebuild/install-samsungtv.sh runs pipx install samsungtvws[cli] (apt pipx if missing). CLI is samsungtv. Skip with SKIP_SAMSUNGTV=1.

Chapter 8 — GNU Radio and SDR (DragonSDR APT_SDR + build libs)

gnuradio, gnuradio-dev, gnuradio-doc, gr-osmosdr, gr-limesdr, gr-fosphor, gr-air-modes, gr-hpsdr, gr-dab, gr-satellites, libsoapysdr-dev, python3-soapysdr, soapysdr-module-osmosdr, soapysdr-module-mirisdr, uhd-soapysdr, rtl-sdr, librtlsdr-dev, airspy, libairspy-dev, bladerf, libbladerf-dev, limesuite, limesuite-udev, uhd-host, libuhd-dev, gqrx-sdr, quisk, inspectrum, hacktv

Chapter 9 — Ham Radio (DragonSDR APT_HAM)

libhamlib-dev, libhamlib-utils, python3-hamlib, fldigi, wsjtx, wsjtx-data, chirp, direwolf, gpredict, grig, xastir, xastir-data

Chapter 10 — HackRF and Mayhem (DragonSDR APT_SDR)

hackrf, hackrf-firmware, libhackrf-dev, hackrf-doc, dfu-util, openocd, gcc-arm-none-eabi, binutils-arm-none-eabi, libnewlib-arm-none-eabi, ccache, lz4, bzip2

EDA — KiCad 10 (APT_KICAD)

Default-on for workstation rebuild. Requires ppa:kicad/kicad-10.0-releases (scripts/rebuild/ensure-kicad-ppa.sh) before install so apt does not pull universe 9.x.

kicad, kicad-libraries, kicad-symbols, kicad-footprints, kicad-packages3d, kicad-templates, kicad-demos, kicad-dbg, kicad-doc-en, kicad-doc-de, kicad-doc-fr, kicad-doc-es, kicad-doc-it, kicad-doc-ja, kicad-doc-pl, kicad-doc-ru, kicad-doc-zh, kicad-doc-ca, kicad-gruvbox-theme, ngspice, gerbv, nodejs, npm

tscircuit (not apt): scripts/rebuild/install-tscircuit.sh runs npm install -g --prefix ~/.local tscircuit @tscircuit/capacity-autorouter typescript. CLI is tsci / tscircuit. Export a circuit

to KiCad with `tsci export circuit.tsx -f kicad_sch` (also `kicad_pcb`, `kicad_zip`). Autorouter npm name is `@tscircuit/capacity-autorouter` (tscircuit-autorouter).

`kicad-doc-id` is **not** in the list (universe still 9.0.8 while the PPA is 10.x). Skip the whole set (including tscircuit) with `SKIP_KICAD=1`.

Chapter 17 — ZFS SMB

`samba` (provides `smbd` and `net` for `zfs share` SMB), `smbclient`. Config: `sudo bin/apply-zfs-smb` (not applied automatically by rebuild).

Chapter 11 — Flatpak

`flatpak` (application `org.telegram.desktop` installed via flatpak, not apt)

Chapters 5, 7, 12, 13 — No dedicated apt arrays

Themes, GNOME prefs, machine utilities, and FactoryDocs use workspace scripts or Ubuntu desktop packages already on the base install.